



COMPUTER SYSTEM LOW LEVEL TECHNIQUES

CT073-3-2-CSLLT

STUDENT NAME	:	HEMANTHRAJAH A/L N KRISHNARAJAH
TP NUMBER	:	TP070844
INTAKE CODE	:	APU2F2411CS(CYB)
ASSIGNMENT	:	INDIVIDUAL ASSIGNMENT
HAND OUT DATE	:	12/05/2025
HAND IN DATE	:	23/07/2025
WEIGHTAGE	:	60%
LECTURER'S NAME	:	NOR AZLINA BINTI ABD RAHMAN

Table of Contents

1.0 Introduction.....	4
1.1 Introduction to Assembly Language and TASM.....	4
1.2 Comparing TASM with MASM and NASM	5
1.3 Why TASM is Ideal for Educational Use	6
2.0 Research and Analysis	8
2.1 Assembly in Embedded and Real-Time Systems	8
2.2 Assembly for System Software, Security, and High-Performance Applications	10
2.2.1 System Startup and Low-Level Infrastructure	10
2.2.2 Device Drivers and Kernel Services	10
2.2.3 Security, Reverse Engineering, and Exploit Development.....	11
2.2.4 High-Performance and Specialized Computation.....	11
2.2.5 Compiler Design and JIT Systems.....	12
2.2.6 UEFI and Firmware Roots of Trust	12
2.3 Assembly in Malware Analysis and Forensics.....	13
2.4 Reverse Engineering Tools: Ghidra and IDA Pro in Practice	14
2.4.1 Ghidra	14
2.4.2 IDA Pro	15
2.4.3 Application in Malware Analysis.....	16
2.5 Bbbloat picoCTF challenge using reverse engineering	17
3.0 Flowcharts.....	24
3.1 Main Menu Flowchart.....	24
3.2 Login Flowchart.....	27
4.0 System Screenshots.....	29
5.0 Source Code with explanation	38

6.0 Conclusion	64
7.0 References.....	66
8.0 Appendices.....	69

1.0 Introduction

1.1 Introduction to Assembly Language and TASM

Assembly language is a low-level programming language that offers direct control over computer hardware through symbolic instructions. Unlike high-level languages that abstract hardware complexity, assembly interacts closely with a machine's architecture, enabling developers to explicitly manage memory, registers, and CPU instructions. This control is crucial in performance-sensitive domains like embedded systems, BIOS development, reverse engineering, and systems programming. Even today, assembly remains foundational for comprehending how software communicates with hardware and for executing optimized, precise code in specialized applications (Tutorials Point, 2021).

Turbo Assembler (TASM), developed by Borland in the late 1980s, remains one of the most user-friendly assemblers for 16-bit and 32-bit x86 development. TASM supports both MASM-compatible syntax and its own structured "IDEAL mode," which enhances code clarity by enforcing explicit brackets and stricter typing rules, ideal for student learning (OSDev, 2023). Bundled with Turbo Linker and Turbo Debugger, TASM creates a cohesive environment for writing, compiling, linking, and debugging, presenting a smooth workflow for academic projects. Its fast compile times and straightforward syntax have made it a staple in educational settings (CliffsNotes, 2021).

A key educational advantage of TASM is the IDEAL mode, which minimizes common syntactical errors and promotes clean code organization. IDEAL mode requires clearly defined directives, such as bracketed memory references and typed registers, helping learners focus on fundamental logic and instruction flow rather than on complex directives. Due to its compatibility with DOS emulators such as DOSBox and its sustained availability through Delphi and C++ Builder, TASM remains a viable tool for academic use even though official support ended around 2010 (OSDev, 2023; CliffsNotes, 2021).

1.2 Comparing TASM with MASM and NASM

While TASM is tailored for educational use, it's valuable to understand its position relative to other dominant assemblers like MASM and NASM. Microsoft Macro Assembler (MASM), in continuous development since 1981, supports 16-, 32-, and 64-bit x86 code with strong integration into Visual Studio and Windows SDKs. Its macro system, structured directives, and Windows API support make it suitable for industrial-strength development, including OS drivers and high-performance modules (Microsoft Docs, 2023).

However, MASM introduces considerable complexity. Completing basic tasks requires explicit memory models, segments, and macros elements unnecessary for fundamental logical operations. This complexity can overwhelm newcomers learning loops, arithmetic, and branching, whose CSLLT project aims to explore. MASM also lacks an integrated lightweight debugger suited for classroom use, instead relying on Visual Studio or similar tools, adding friction to initial learning phases (Dahlquist, 2021).

In contrast, Netwide Assembler (NASM), launched in 1996, is a modern, open-source assembler known for its strict and portable syntax. It enforces explicit operand sizes (e.g., BYTE, WORD, DWORD) and bracketed memory references, rejecting ambiguous directives like ASSUME or DUP. NASM supports multiple output formats like ELF, COFF, Mach-O, and raw binaries making it a powerful tool for OS development and cross-platform work (NASM Documentation, 2022).

Nevertheless, NASM's stringent syntax and reliance on standalone tools like ld and gdb create steep initial setup for beginners. Though its explicit requirements help reinforce good coding habits, they also introduce verbosity and complexity, which can be detrimental to those seeking to grasp core assembly logic quickly (NASM posix doc, 2023).

Despite sharing the same core instruction set, TASM, MASM, and NASM differ significantly in syntax, tooling, and use cases. TASM provides simplicity and immediacy for beginners; MASM

supports enterprise-level Windows development; NASM serves open-source, cross-platform, and systems-level programming. For your CSLLT project, which emphasizes mastering assembly fundamentals like arithmetic operations and control flow, TASM offers the most frictionless environment.

1.3 Why TASM is Ideal for Educational Use

TASM's educational strengths manifest in several key areas:

1. **IDEAL Mode for Clarity**

In IDEAL mode, bracketed memory references and typed registers are mandatory, eliminating ambiguities and improving comprehension. OSDev documentation acknowledges IDEAL mode as a cleaner, more consistent syntax than MASM's, thus preventing errors earlier in the development cycle (OSDev, 2023).

2. **Faster Compilation**

IDEAL mode runs up to 30% faster than MASM-compatible mode, rounding up quick iteration cycles is essential for classroom assignments involving debugging and repeated testing (CliffsNotes, 2021).

3. **Integrated Toolchain**

Unlike MASM or NASM, which require multi-tool setups, TASM includes Turbo Debugger and Turbo Linker. This unified workflow allows students to edit, assemble, link,

and debug without switching contexts, facilitating immediate feedback while tracing registers, flags, and stack operations (WPI CS, 2021).

4. Legacy and Protected Mode Support

TASM supports both DOS-style 16-bit and 32-bit protected-mode code via TASM32.EXE, enabling students to explore a wider breadth of x86 programming without added complexity. DOSBox compatibility ensures modern usability.

5. Comprehensive Educational Ecosystem

TASM is still supported by numerous tutorials, manuals, and university course materials. CliffsNotes even highlights its suitability for small-scale educational programs, emphasizing its speed and clarity for student use (CliffsNotes, 2021). OSDev FAQ confirms TASM's ongoing relevance in educational circles (OSDev, 2023).

By starting with TASM in IDEAL mode, learners can develop a firm understanding of control structures like loops and conditional jumps. This foundation can later be expanded with MASM for Windows API and advanced macros, or with NASM for low-level OS and cross-platform programming. This staged approach ensures students master key logic constructs before tackling more advanced assembler features.

2.0 Research and Analysis

2.1 Assembly in Embedded and Real-Time Systems

Assembly language remains essential in **embedded and real-time systems** where hardware control, resource constraints, and timing precision are critical. Devices like microcontrollers, IoT sensors, automotive modules, and medical instruments often rely on hand-tuned assembly for performance and reliability (AllAboutCircuits, 2022).

Many embedded controllers operate under tight memory, power, and real-time constraints that require deterministic behavior. Assembly allows developers to write compact code that interacts directly with hardware peripherals such as timers, ADCs, and UARTs. For example, interrupt service routines (ISRs) written in assembly execute with minimal latency and precise cycle timing, something essential in motor control, safety systems, and sensor data acquisition (AllAboutCircuits, 2022; The Pi Guy, 2024). The Pi Guy also emphasizes that writing ISRs in assembly ensures consistent and fast hardware responses.

Even though high-level languages like C and Rust are often used for most embedded tasks, assembly continues to play a crucial role in low-level components. Bootloaders, startup routines, and memory initialization sequences are commonly written in assembly to guarantee that the system can correctly enter protected or real-time modes (Embedded.com, 2023). For instance, Cortex-M microcontrollers typically begin execution in an assembly startup file before control is transferred to C code (DragonZap, 2023).

Developers working in environments like FreeRTOS or bare-metal firmware frequently blend high-level code with inline or standalone assembly. They often embed small assembly routines for operations like context switching, bitfield manipulation, or read/write of memory-mapped registers. These routines are compact, sometimes just a handful of instructions and carry out tasks that would be inefficient or unpredictable in C (AllAboutCircuits forum, 2023; Frosnerd, 2020). Using assembly in these areas enables precise hardware control, minimizes stack usage, and

guarantees predictable execution times, qualities modern compilers and real-time OSes alone cannot reliably provide.

The necessity of assembly in embedded systems becomes clear in safety-critical or resource-constrained domains. For example, in aerospace, medical devices, or automotive platforms, system failures can have grave consequences. Detailed micro-benchmarks and certifications often drive developers to use assembly in parts of their code that require certified performance or verified timing. While high-level languages facilitate maintainability, assembly ensures deterministic behavior and compliance with timing budgets.

In addition, several studies highlight how embedding assembly within C improves performance and precision. One article notes that many developers still prefer to write ISRs and vertical interrupt handlers directly in assembly to avoid compiler uncertainty and to precisely manage stack pointer movements and register use (AllAboutCircuits forum, 2023). Similarly, the Cortex-M bootloader and hardware initialization steps at power-on are managed using assembly to ensure consistent behavior and reliable transfer to high-level firmware (AllAboutCircuits, 2022; DragonZap, 2023).

Educational practice also reflects this mixed approach. Many university labs teach students to write a minimal bootloader for microcontrollers using assembly and then transition to C for system logic, building a foundational understanding of hardware interaction and abstraction (UNC COMP730 lab, 2025). This approach equips students to make informed decisions about where assembly is necessary and when high-level code is sufficient.

Overall, while less code is written in assembly now compared to decades ago, certain critical components such as bootloaders, interruption handlers, memory initialization, and performance-critical routines still rely on it heavily. Even when most code is written in C or Rust, assembly remains a strategic tool for ensuring timing reliability and direct hardware interaction in embedded systems.

2.2 Assembly for System Software, Security, and High-Performance Applications

Beyond embedded systems, assembly plays a vital role in **system software, security tasks, and performance-critical applications**. Operating-system kernels, bootloaders, device drivers, security tools, and optimized computational routines often include assembly to meet precise hardware requirements first and foremost (MoldStud Research, 2025; DragonZap, 2023).

2.2.1 System Startup and Low-Level Infrastructure

Bootloaders are the first code that runs on a machine, initializing hardware and loading the operating system. These stages include setting up the Global Descriptor Table (GDT), switching CPU modes, and configuring memory all typically implemented in assembly (DragonZap, 2023; MoldStud Research, 2025). For example, tutorials on x86 bootloaders show how BIOS loads 512 bytes at address 0x7C00, and how assembly sets up segment registers and stack frames before launching the kernel (DragonZap, 2023). In ARM systems, firmware uses assembly to configure exception vectors, memory controllers, and startup configurations prior to executing C or Rust code (DragonZap, 2023).

Many modern bootloader codebases like Limine and Barebox continue to leverage assembly for the early initialization layer before switching to C (MoldStud Research, 2025). MoldStud's article confirms that these routines must be executed without runtime support and rely on precise, direct register manipulations best handled in assembly.

2.2.2 Device Drivers and Kernel Services

Within operating-system kernels, some modules still contain assembly for critical performance or hardware control. This includes context-switch routines, interrupt dispatchers, and peripheral drivers that require direct CPU register access or atomic synchronization. Writing these parts in assembly ensures consistency and efficiency across hardware platforms, especially where compilers may generate less optimal instructions or make unsafe assumptions (AllAboutCircuits, 2022).

Kernel developers often resort to inline or standalone assembly in C when implementing atomic operations, port I/O instructions, or specialized instructions (e.g., CPUID, RDTSC). These routines are low in volume but high in impact, demanded by modern CPU architectures that maintain strict semantics around memory ordering, atomicity, and system transition points.

2.2.3 Security, Reverse Engineering, and Exploit Development

In cybersecurity and vulnerability analysis, assembly knowledge is foundational. Malware analysts and reverse engineers disassemble binaries to understand control flow, encryption logic, and hidden behaviors such as vulnerability exploitation or anti-debug techniques. Open-source tools like IDA Pro or Ghidra visualize machine code and require analysts to recognize opcodes and calling conventions quickly (MoldStud Research, 2025).

Assembly language is also used in writing shellcode or exploit payloads, where size and obfuscation matter. Shellcode often exploits buffer overflows or ROP chains with only a few dozen bytes, crafted in handwritten assembly to avoid detection and achieve precise control over the execution environment. Though the share of malware written entirely in assembly is small, the most critical pieces payloads and jump-to-shellcode routines are handwritten in assembly to evade sandboxes and injection filters.

2.2.4 High-Performance and Specialized Computation

Many applications with stringent performance requirements include assembly for their core computations. This includes cryptographic libraries, scientific simulations, signal processing, and high-frequency trading systems areas that depend on predictable performance or microsecond-level speed gains (MoldStud Research, 2025). For instance, video encoding libraries like AV1 frequently include assembly to utilize SIMD extensions (AVX2, SSE, Neon) that compilers cannot automatically vectorize (MoldStud Research, 2025; AllAboutCircuits, 2022).

Similarly, high-frequency trading requires ultra-low latency in message handling and protocol processing. Firms often hand-write time-critical network stacks, logging, or serialization routines in assembly to remove branches and ensure cache-friendly execution (Bilokon & Lucuta, 2023).

Bloom filters, hash functions, and memory-parallel algorithms are also prime candidates for inline assembly to optimize throughput and latency.

2.2.5 Compiler Design and JIT Systems

Compiler backends and just-in-time (JIT) engines often generate dynamic machine instructions, especially when implementing runtime profiling, dynamic specialization, or inline caching. This requires precise knowledge of calling conventions, stack alignment, register use, and CPU features often producing snippets of machine code via assembly templates or serialized binary sequences (AllAboutCircuits, 2022). Successful JIT backends, such as those used in JavaScript engines, rely on well-crafted assembly sequences for core loops and trampolines.

2.2.6 UEFI and Firmware Roots of Trust

Modern firmware platforms like UEFI and Trusted Computing Environments are not exception. The UEFI Security (SEC) phase, which runs before memory is fully initialized, is written in assembly to set up cache-as-RAM and firmware integrity checks (UEFI spec, 2025). This routine ensures a trusted boot sequence before handing off to higher-level firmware components (UEFI spec, 2025).

2.3 Assembly in Malware Analysis and Forensics

Assembly language remains foundational in **malware analysis** and digital forensics across cybersecurity domains. By inspecting malware binaries at the instruction level, analysts unravel obfuscated logic, uncover hidden functionality, and reverse engineer encryption routines. IBM's Security blog emphasizes that understanding assembly is essential when the source code is unavailable, assembly represents the only translation of a program that analysts can study to determine malicious behavior (Puckett, 2022). With assembly-level examination, researchers can trace precise sequences of instructions such as file encryption loops, registry modifications, or network communication logic that define malware behavior.

Static analysis tools such as **Ghidra** and **IDA Pro** transform compiled binaries into readable assembly code, enabling analysts to identify control flow, function boundaries, and API calls. Ghidra, being open-source and free, has become especially popular since its 2019 release, allowing malware researchers to perform decompilation, scripting, and collaborative work (Varonis, 2023). Assembly inspection often reveals important artifacts such as malicious string constants, encryption keys, and system calls that help build effective detection signatures.

Dynamic analysis complements static inspection by using debuggers like **WinDbg** or **OllyDbg**, enabling analysts to execute malware in a sandboxed environment and trace runtime behavior. Analysts can step through critical assembly instructions to observe how malware resolves API addresses, decrypts payloads in memory, or alters control flow. Forensic investigators rely on this low-level visibility to classify novel malware variants and to design countermeasures such as IPS signatures or cleanup routines (BugBustersUnited, n.d.).

Recent studies underscore how control-flow features extracted from assembly, such as relative jumps or branch instruction patterns, can feed statistical classifiers to detect malware automatically (Linke & Le-Khac, 2016). While such research may not be from 2022, the principles apply continuously assembly remains the substrate for feature extraction in machine-learning approaches to malware detection.

2.4 Reverse Engineering Tools: Ghidra and IDA Pro in Practice

Two standout tools in modern reverse engineering and staples of forensic analysis are **Ghidra** and **IDA Pro**, both indispensable when working with assembly code.

2.4.1 Ghidra



Figure 1: Ghidra logo

Developed by the NSA and open-sourced in 2019, Ghidra offers comprehensive reverse-engineering features including a built-in decompiler, scripting capabilities in Python and Java, and real-time project collaboration (Kenji, 2024; Varonis, 2023). Its decompiler translates assembly into readable pseudo-C code, easing the comprehension of malware logic and enabling analysts to identify loops, conditional branches, and data structures. Analysts can navigate assembly listings using Ghidra's Code Browser, rename functions, inspect imports/exports, and visualize control flow graphs (Kenji, 2024).

Ghidra is widely used to analyze malware safely in sandbox environments, as it disassembles binaries without executing them. Analysts can trace call sequences, examine obfuscated logic, and locate suspicious functions such as encrypted payload loaders or registry writers by cross-referencing assembly instructions with network or file behavior indicators (Lepide, 2024). Its scripting support helps automate common patterns, such as identifying API calls or renaming symbols, which speeds up forensic workflows.

Because Ghidra is free, it democratizes access to reverse-engineering. Medium-based tutorials and cybersecurity platforms have popularized it as a beginner-friendly yet powerful SRE suite (Kenji, 2024). Incident responders and malware researchers often leverage Ghidra in multi-analyst setups, using its built-in collaboration server to share findings during live investigations.

2.4.2 IDA Pro



Figure 2: IDA PRO logo

IDA Pro remains a gold standard in professional reverse engineering, offering robust disassembly, graph-based navigation, and optional decompiler plugins. It supports a wide range of executable formats and CPU architectures, making it suitable for analyzing malware targeting Windows, macOS, Linux, or firmware binaries (IDA Pro, 2025). IDA Pro's interactive graphs and signature recognition features help analysts quickly map high-level control structures and identify familiar library functions within assembly code.

While Ghidra and IDA overlap in functionality, IDA Pro has historically focused on polished UI and mature features. It integrates binary analysis and debugging, allowing analysts to view assembly instructions alongside cross-references and annotate control flow logic. Many incident

response teams continue to use IDA Pro for advanced threat analysis, vulnerability research, and forensic casework especially in closed-source enterprise environments.

2.4.3 Application in Malware Analysis

In practice, security professionals use these tools to examine malicious binaries. For example, after capturing a ransomware sample, an analyst might import it into Ghidra to identify the encryption loop and halt conditions in assembly. They might then use IDA Pro to graphically visualize branch logic or confirm obfuscation layers. Analysts trace how the malware resolves API addresses at runtime using dynamic assembly insight via debuggers, they can then reproduce these behaviors in an emulated sandbox or craft detection routines.

A powerful pattern in malware analysis is the creation of normalized intermediate representations. Frameworks such as MAIL (Malware Analysis Intermediate Language) abstract away architecture specifics to focus on features like opcode sequences and control-flow transitions again built on assembly-level extraction (Alam, 2022). These tools illustrate the symbiosis between assembly codes, automated analysis, and manual reverse engineering.

2.5 Bbbloat picoCTF challenge using reverse engineering

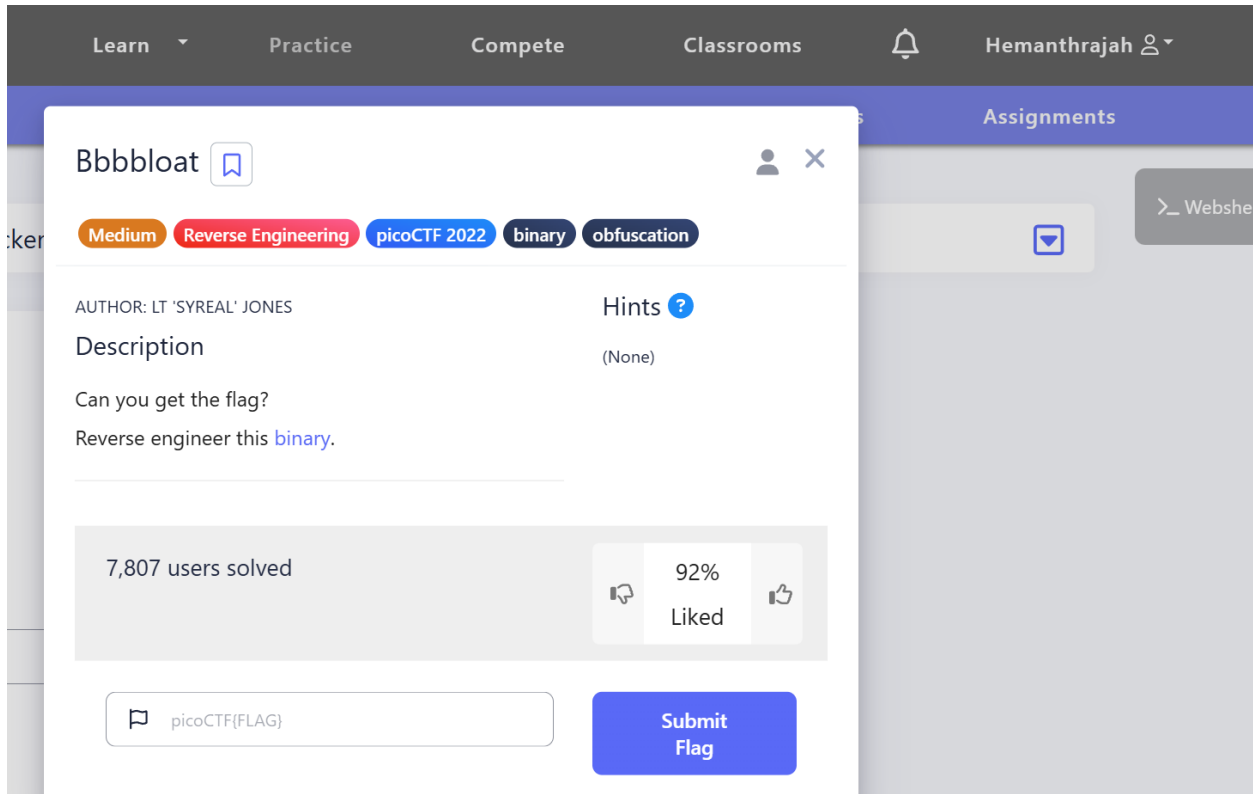


Figure 3: Bbbloat picoCTF challenge

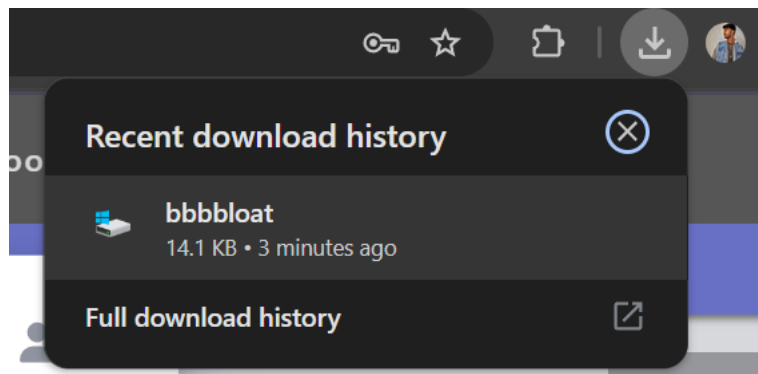


Figure 4: Downloaded bbbloat file

I downloaded the Bbbloat file from the the challenge in picoCTF. It had was medium difficulty and had a good rating where 92% of the users liked the challenge.

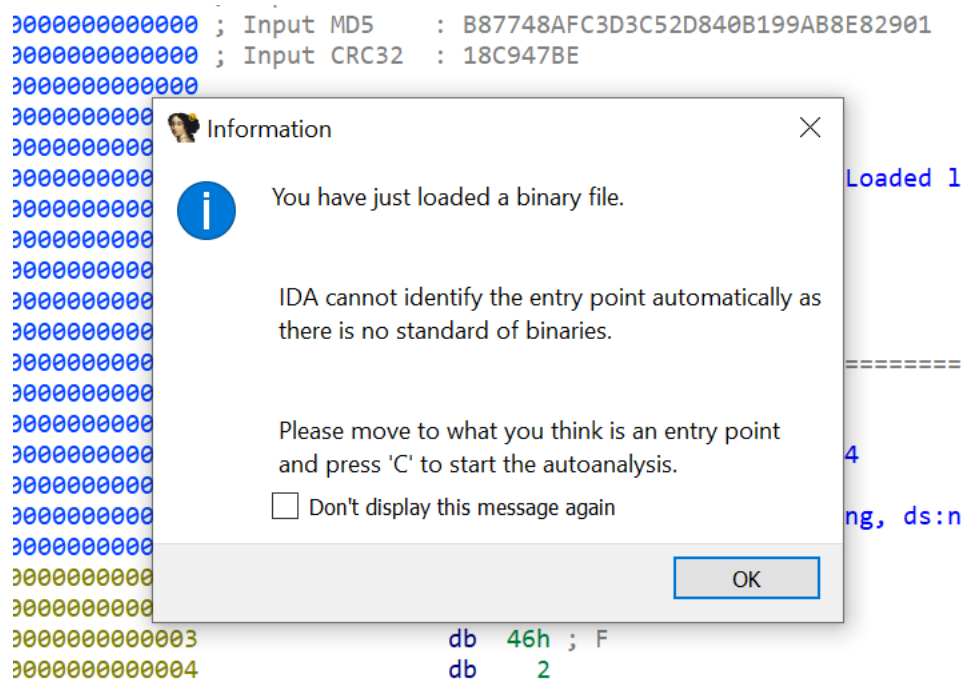


Figure 5: Failed dissembling in IDA free 9.1

I first tried disassembling the file using IDA free 9.1 but it couldn't identify the entry point, so I proceeded to use Ghidra.

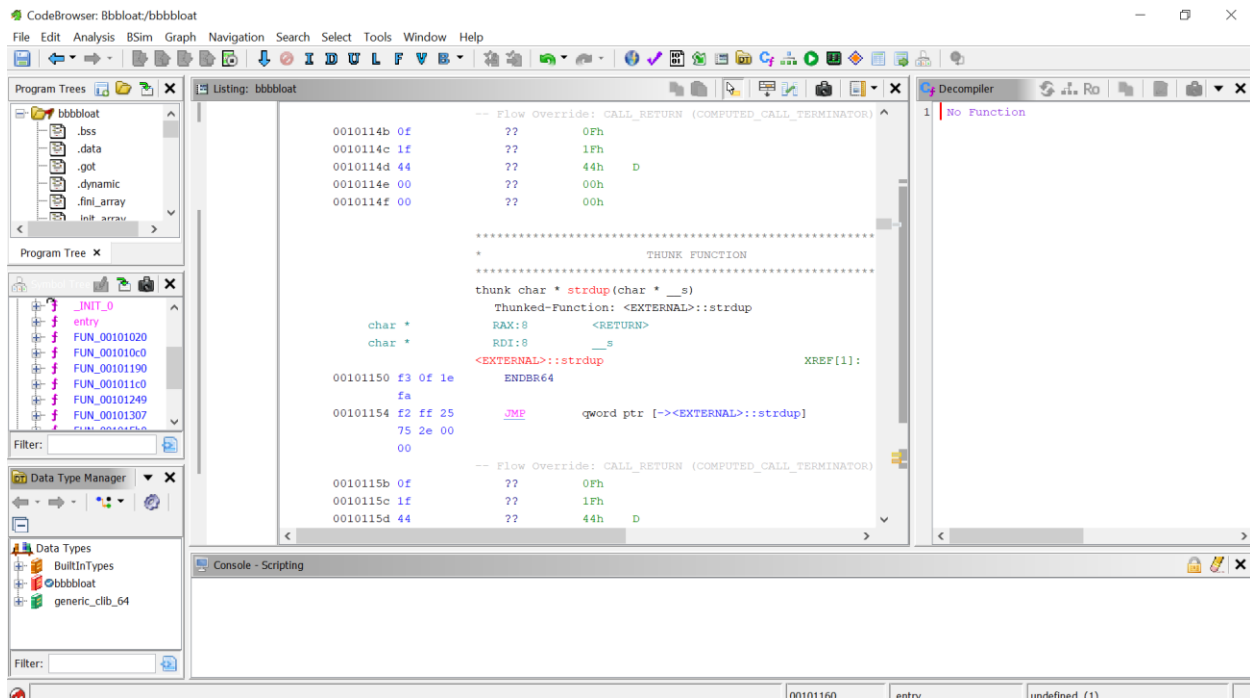


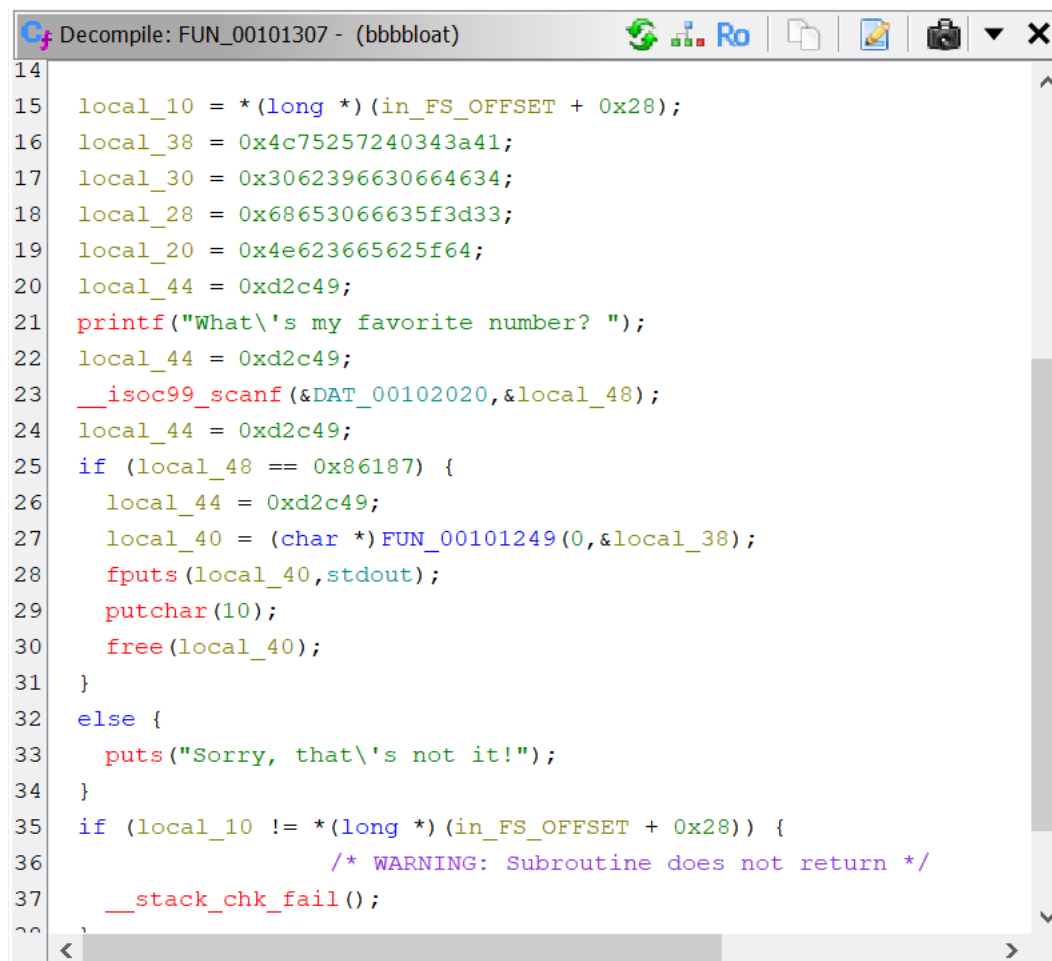
Figure 6: Disassembled bbbloat binary file in Ghidra

When I successfully loaded the "Bbbloat" executable into Ghidra, my primary goal was to dissect its inner workings and understand where the "bloat" was truly hidden. After the initial auto-analysis, I started by navigating to the main function, which is always my starting point to get a high-level overview of the program's execution flow.

As I began to explore, I quickly encountered a function, which Ghidra labeled something like FUN_00101290, that immediately stood out. Looking at this screenshot, this is exactly the kind of "bloat" the challenge title hinted at! In the Decompiler window on the right, I could clearly see a seemingly endless series of if statements. Each if statement was performing a comparison: it was checking if an input parameter, which Ghidra named param_1, was equal to a very specific, large hexadecimal constant (like 0x83e87de7, 0x8e815615, 0x1f06a86f, and so on). If any of these conditions were met, the function would simply return 0x1; otherwise, it would eventually return 0x0. This immediately told me that this function was acting as a massive validation gate or a lookup table, designed to accept only one of many possible "correct" inputs.

In the Disassembly Listing, right next to the decompiled code, I could observe the direct assembly instructions that correlated with these if checks. I saw a repeating pattern of CMP (compare) instructions followed by JNE (jump if not equal) or JE (jump if equal) instructions. This confirmed

my understanding from the pseudocode: the program was sequentially comparing the input against a long list of hardcoded values. My strategy then became clear: I needed to figure out what was calling this "bloated" function and, more importantly, what value it was actually looking for among these many comparisons. This function itself wasn't directly revealing the flag, but it was a crucial hurdle that needed a specific input to bypass, leading me closer to the actual solution of the "Bbbloat" challenge.



```
Decompile: FUN_00101307 - (bbbbloat)
14
15  local_10 = *(long *) (in_FS_OFFSET + 0x28);
16  local_38 = 0x4c75257240343a41;
17  local_30 = 0x3062396630664634;
18  local_28 = 0x68653066635f3d33;
19  local_20 = 0x4e623665625f64;
20  local_44 = 0xd2c49;
21  printf("What\'s my favorite number? ");
22  local_44 = 0xd2c49;
23  __isoc99_scanf (&DAT_00102020,&local_48);
24  local_44 = 0xd2c49;
25  if (local_48 == 0x86187) {
26      local_44 = 0xd2c49;
27      local_40 = (char *) FUN_00101249(0,&local_38);
28      fputs(local_40,stdout);
29      putchar(10);
30      free(local_40);
31  }
32  else {
33      puts("Sorry, that\'s not it!");
34  }
35  if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
36      /* WARNING: Subroutine does not return */
37      __stack_chk_fail();
38  }
```

Figure 7: FUN_00101307 function

As I continued to reverse engineer the "Bbbloat" binary in Ghidra, systematically shifting through various functions, I eventually came across a very interesting one that Ghidra identified as FUN_00101307. This function immediately caught my attention because it seemed to be a core part of a crucial validation step, effectively cutting through some of the initial "bloat" I'd observed.

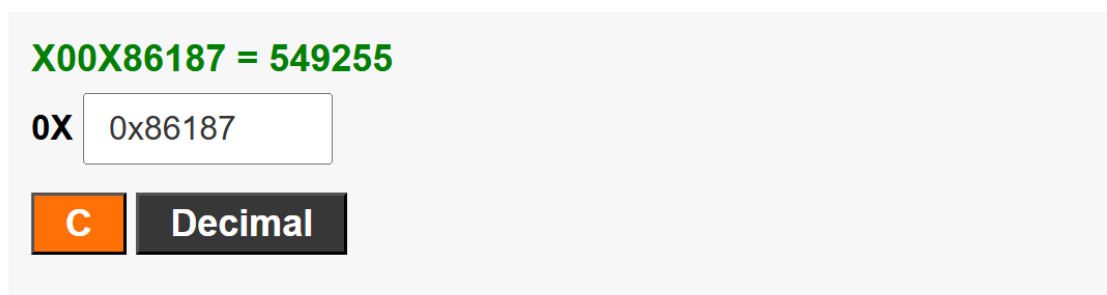
Upon inspecting the decompiled C code for FUN_00101307, I noticed a clear prompt: `printf("What\\'s my favorite number? ");`. This was a direct signal that the program was expecting some form of numerical input. Immediately following this, I observed the `_isoc99_scanf` call, which confirmed that the program was reading user input into a local variable, `local_48`. The critical breakthrough in this section came with the subsequent if statement: `if (local_48 == 0x86187) { ... }`. This line explicitly showed that the program was comparing my input directly against the hexadecimal value `0x86187`. This was the specific "favorite number" that the challenge was looking for to proceed.

Understanding this conditional check was pivotal. It meant that if I provided `0x86187` as the input, the program would take the success path. In this path, I could see it calling another function, `FUN_00101249`, before using `fputs` and `putchar`. This suggested that `FUN_00101249` was likely responsible for generating or revealing the actual flag or a crucial part of it. Conversely, if my input didn't match, the else branch would simply print "Sorry, that's not it!". Identifying this exact numerical comparison was the decisive step in bypassing this specific "bloated" validation gate and advancing towards solving the picoCTF Bbbbloat challenge.

[Hex to Decimal Converter](#)

Here you can convert another hexadecimal number to decimal.

Remember hex numbers include numbers 0 through 9 and letters A through F.



X00X86187 = 549255

0X

C **Decimal**

Figure 8: Hex to Decimal Converter online

After identifying that `FUN_00101307` was looking for the specific hexadecimal value `0x86187`, my next step was to convert this hexadecimal number into its decimal equivalent. Since the

program's prompt asked for a "favorite number" and typically user input is provided in decimal form unless otherwise specified, converting it was a necessary step to give the correct input. I used an online "Hex to Decimal Converter" for this purpose. As you can see in the screenshot, inputting 0x86187 into the converter yielded the decimal value 549255. This decimal number, 549255, was then the exact value I needed to provide to the "Bbbbloat" executable to satisfy that specific check and proceed further in the challenge.

```
Hemanthraja-picoctf@webshell:~$ wget https://artifacts.picoctf.net/c/46/bbbbloat
--2025-07-21 17:36:50-- https://artifacts.picoctf.net/c/46/bbbbloat
Resolving artifacts.picoctf.net (artifacts.picoctf.net)... 3.160.22.128, 3.160.22.92, 3.160.22.16, ...
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|3.160.22.128|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14472 (14K) [application/octet-stream]
Saving to: 'bbbbloat'

bbbbloat                               100%[=====>] 14.13K  --.-KB/s  in 0s
2025-07-21 17:36:50 (487 MB/s) - 'bbbbloat' saved [14472/14472]
```

Figure 9: picoCTF webshell

Then, I leveraged the picoCTF webshell, which provides a convenient command-line environment for interacting with challenge files. As shown in the screenshot, I used the `wget` command, specifying the provided URL: `https://artifacts.picoctf.net/c/46/bbbbloat`. This command directly downloads the file from the remote server to my current working directory within the webshell. The output confirmed that `bbbbloat` was successfully saved, indicating that I now had the executable ready for further analysis and to eventually run it to test my findings.

```
Hemanthraja-picoctf@webshell:~$ chmod u+x ./bbbbloat
Hemanthraja-picoctf@webshell:~$ ./bbbbloat
What's my favorite number? 549255
picoCTF{cu7_7h3_bl047_695036e3}
Hemanthraja-picoctf@webshell:~$
```

Figure 10: Flag Bbbbloat picoCTF challenge

Once I had the `bbbbloat` binary on the PicoCTF webshell, I knew my next step was to make it executable so I could run it. As you can see in this screenshot, I used the `chmod u+x ./bbbbloat` command to grant execute permissions to the file.

After successfully setting the permissions, I executed the binary by typing `./bbbbloat`. The program then prompted me with the question, "What's my favorite number?". Thanks to my reverse engineering efforts in Ghidra, where I found the specific hexadecimal value `0x86187` that the program was looking for, and my subsequent conversion of that hex number to decimal, I knew the exact input required. I promptly entered `549255` as the favorite number. Upon providing the correct input, the program immediately revealed the flag: `picoCTF{cu7_7h3_b1o47_695036e3}`. This was the final piece of the puzzle, confirming that my analysis of the "bloated" function and the numeric comparison was accurate.

3.0 Flowcharts

3.1 Main Menu Flowchart

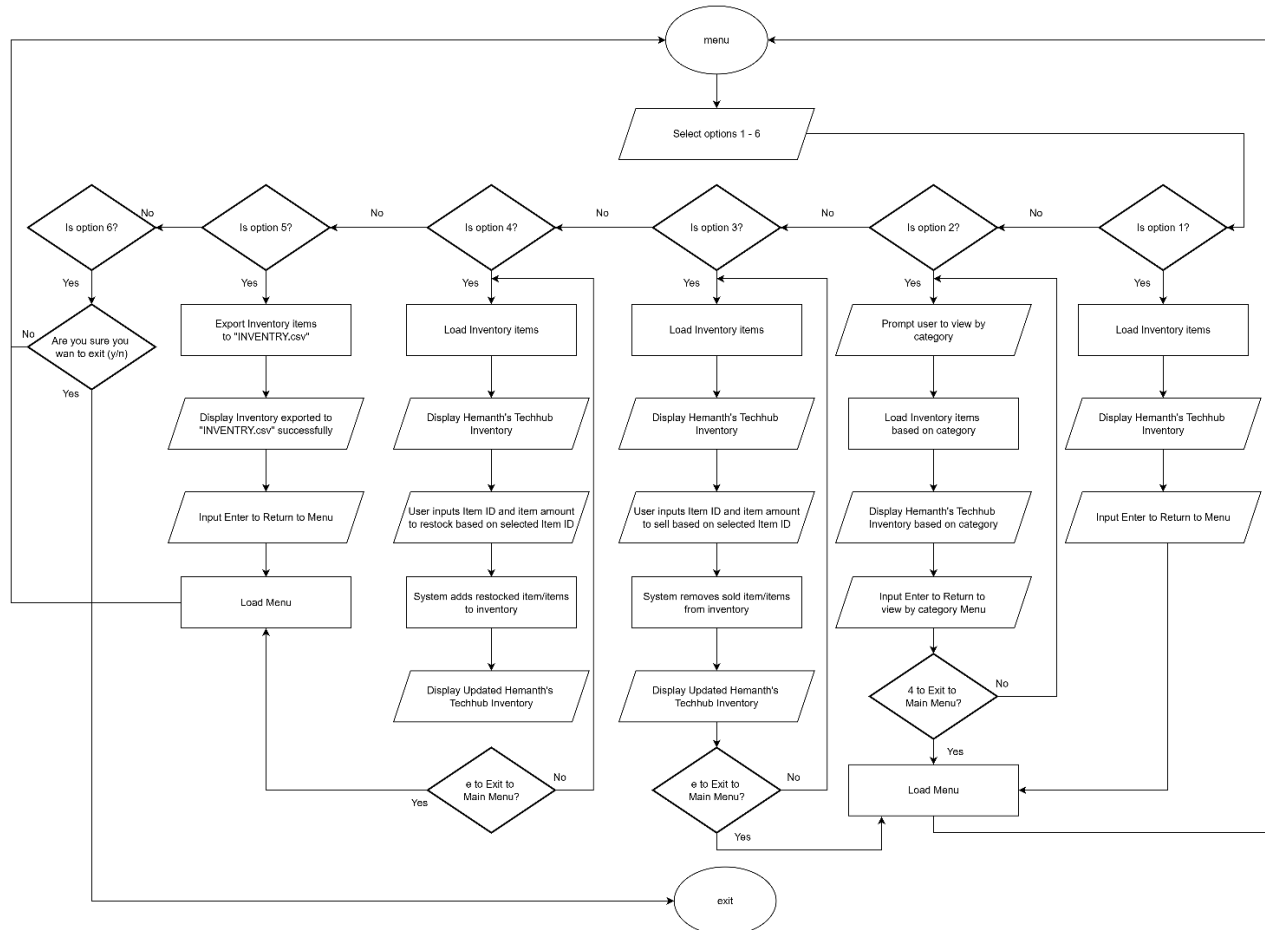


Figure 11: Main Menu Flowchart

This flowchart outlines the operational flow of my "Hemanth's Techhub Inventory System," as visually represented in the accompanying flowchart. My system is designed with a clear, menu-driven interface to facilitate intuitive inventory management for the user.

At the core of my system's user interaction is the main menu. This is where everything begins after successful authentication. When the program runs, I present the user with a set of numbered

options, typically from 1 to 6, each corresponding to a specific inventory management task. My system patiently waits for the user to input their choice.

Once a selection is made, my system meticulously processes the input through a series of conditional checks to determine which functionality the user wishes to access:

If the user selects Option 1, which is "View All Inventory Items," my system immediately proceeds to load all the current inventory data. After this data is retrieved, I display it comprehensively on the screen, giving the user a complete overview of every item in stock. Following the display, I prompt the user to press the Enter key, allowing them to conveniently return to the main menu and select another task.

Should the user opt for Option 2, "View Inventory by Category," my system first prompts them to specify which category they are interested in viewing. Based on this input, I then load and display only the inventory items that fall under that category. This provides a focused view for the user. After presenting the filtered list, I again prompt for input to bring them back to the main menu, ensuring a smooth transition.

When Option 3, "Sell Inventory Items," is chosen, my system begins by displaying the current full inventory, so the user can see what's available. Next, I guide the user to input the specific Item ID and the quantity of the item they intend to sell. Upon receiving this information, my system processes the transaction by deducting the sold amount from the inventory records. I then display the updated inventory, confirming the sales and reflecting the sales stock levels. Finally, I offer the user an opportunity to return to the main menu, ready for more sales or other operations.

For Option 4, "Restock Inventory Items," my system mirrors the initial steps of the selling process by loading and displaying the current inventory. Following this, I prompt the user to provide the

Item ID and the quantity of the items they wish to add back into stock. My system then performs the necessary update, increasing the inventory count for the specified item. To ensure transparency, I display the newly updated inventory. Like other operations, I conclude by prompting the user to return to the main menu, facilitating continuous restocking operations if needed.

If the user selects Option 5, "Export Inventory Data," my system executes a dedicated function to save the entire current inventory database. This data is written into an external file, which I specifically name "INVENTORY.csv." Once the export process is successfully completed, I provide a clear confirmation message to the user. My system then waits for the user to acknowledge by pressing Enter, at which point I navigate them back to the main menu.

Finally, when Option 6, "Exit Application," is chosen, my system incorporates a vital safeguard: I prompt the user for confirmation, asking if they are truly sure they want to exit (e.g., "Are you sure? (y/n)"). If the user provides an affirmative response, my system proceeds to terminate. However, if they indicate they do not wish to exit, I gracefully return them to the main menu, allowing them to continue using the application without interruption.

This detailed, option-driven structure ensures that my Hemanth's Techhub Inventory System is both user-friendly and robust, allowing for efficient and controlled management of inventory tasks.

3.2 Login Flowchart

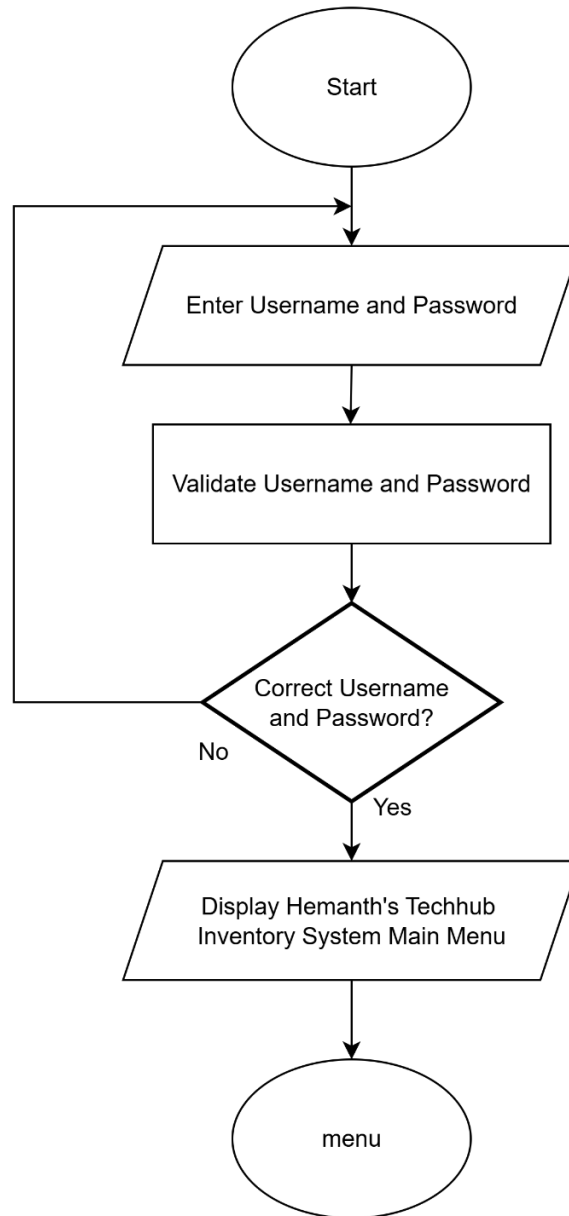


Figure 12: Login Flowchart

This flowchart outlines the login process for my "Hemanth's Techhub Inventory System," as illustrated in the accompanying flowchart. My primary goal with this segment of the system is to ensure secure and controlled access to the inventory management functionalities.

The login sequence begins at the Start point of my flowchart. Immediately, my system prompts the user to Enter Username and Password. This is where the user provides their credentials for authentication.

Once the username and password have been input, my system proceeds to Validate Username and Password. This is a crucial internal process where I compare the entered credentials against the stored, correct username and password.

Following the validation, my system reaches a pivotal decision point: Correct Username and Password? Here, I determine if the provided credentials match the expected values.

- 1) If the credentials are incorrect ("No" branch): My system loops back to the "Enter Username and Password" step. This means the user is given another opportunity to input their credentials, preventing immediate lockout due to a simple typo and reinforcing the need for correct authentication.
- 2) If the credentials are correct ("Yes" branch): My system grants access. The next step is to Display Hemanth's Techhub Inventory System Main Menu. This indicates a successful login, and the user is now presented with the primary interface for managing the inventory.

Finally, after the main menu is displayed, the flow transitions to a "menu" state, which signifies the entry point into the broader menu system that I explained previously. This establishes a seamless connection between the login process and the main operational menu of my inventory system. This entire login mechanism ensures that only authorized users can access and manage

4.0 System Screenshots

```
tlink      -      Perform Linking
td         -      Launch Turbo Debugger
Exit       -      Exit Tasm 1.2

-----
      For Compiling your files tasm "yourfilename".asm use without quotes
e.g for compiling add.asm command is : tasm add.asm
For Linking and debugging same as 32 bit : tlink,td
      .
      Complink,DPMIload and TasmX also available using 32bit commands
-----

C:\TASM>Tasm TechhubH.asm
Turbo Assembler  Version 3.0  Copyright (c) 1988, 1991 Borland International

Assembling file:   TechhubH.asm
Error messages:    None
Warning messages:  None
Passes:            1
Remaining memory:  421k

C:\TASM>Tlink TechhubH.obj
Turbo Link  Version 2.0  Copyright (c) 1987, 1988 Borland International

C:\TASM>TechhubHS_
```

Figure 13: Compiling assembly code

In the process of developing my assembly language program, I first input my TechhubH.asm source code into the Turbo Assembler (TASM). This step compiles the human-readable assembly instructions into a machine-executable object file (TechhubH.obj), effectively translating my code into a format the computer can understand. Following this, I input TechhubH.obj into the Turbo Linker (TLINK), which resolves any external references and combines all necessary code segments to produce my final executable program, TechhubH.exe, ready for direct execution on the system.



```
Enter Username: Hemanth_
```

Figure 14: Username



```
Password for Hemanth's Techhub Inventory System : Hemanth1234
```

Figure 15: Password

My system then prompts me to enter the username and password for my "Hemanth's Techhub Inventory System,". If the username and password is valid, the system will go into the Main Menu.



```
-----<Hemanth's Techhub Inventory System>-----  
-----MAIN MENU-----  
1.View Storage Inventory  
2.View Items By ... Category  
3.Sell Electronic Devices or Accessories  
4.Items Restock  
5.Export Inventory  
6.Exit the Program  
Select an Option: _
```

Figure 16: Main Menu

My main menu is designed for clarity and ease of use. At the very top, I've prominently placed the system title, "Hemanth's Techhub Inventory System," clearly indicating to the user which application they are interacting with. Below this, I've centered a sub-header, "MAIN MENU," to explicitly define the current navigation point. The core of the menu is a vertically aligned, numbered list of six options. This sequential numbering allows for quick visual scanning and direct selection by the user. Each option is a self-explanatory label, telling the user exactly what action

selecting that number will perform. Finally, I conclude the display with a simple "Select an Option:" prompt, guiding the user to input their choice.

In terms of what it does, this menu serves as the central dispatcher for all major operations within my inventory system.

- Option 1, "View Storage Inventory," allows the user to see every item currently recorded in the inventory.
- Option 2, "View Items By ... Category," provides a filtered view, letting the user narrow down the inventory list by specific item types.
- Option 3, "Sell Electronic Devices or Accessories," initiates the process for deducting items from stock after a sale.
- Option 4, "Items Restock," is for adding new quantities of existing items back into the inventory.
- Option 5, "Export Inventory," enables the user to generate an external CSV file containing the complete inventory data for record-keeping or further analysis.
- And finally, Option 6, "Exit the Program," provides a clean and confirmed way for the user to gracefully close the application.

Essentially, I've structured this menu to be a straightforward command center, making all the primary inventory management tasks easily accessible.

-----<Hemanth's Techhub Inventory System>-----			
-----<INVENTORY>-----			
ID	Name	Price	Quantity
1	Samsung Galaxy Tab S9	RM4999	4
2	iPhone 16 Pro Max	RM7999	6
3	iPad Air 6th Gen	RM3899	1
4	Sony Bravia XR A95L	RM15999	8
5	Google Pixel 9 Pro	RM4799	3
6	Apple Watch Series X	RM1899	7
7	Samsung Soundbar Q990D	RM3299	2
8	Xiaomi Pad 7	RM1699	5
9	Samsung Galaxy Z Fold 6	RM7899	4
Press Enter to Return to MenuS			
Press Enter to Return to Menu			

Figure 17: View Storage Inventory

This is a clear view of the "View Storage Inventory" screen within my "Hemanth's Techhub Inventory System," which is accessed by selecting Option 1 from the main menu. I have designed this layout to be highly functional and easy to read, presenting all relevant inventory data in a structured tabular format. The display begins with a consistent system header, followed by a prominent "INVENTORY" sub-header, immediately informing the user of the content. Below this, I have established distinct columns: "ID," "Name," "Price," and "Quantity," ensuring that each piece of information is clearly categorized. Each item is then listed on its own line, with its unique ID, product name, price, and current quantity neatly aligned under the respective headers. Notably, I've implemented a visual cue where quantities of 5 or less are highlighted in red, providing a quick visual alert for low stock. After presenting the entire inventory, my system prompts the user to "Press Enter to Return to Menu," ensuring a seamless return to the main operational hub. This screen's primary function is to give the user a complete and immediate overview of all items currently in stock, facilitating quick checks and informed decisions.

-----<Hemanth's Techhub Inventory System>-----	
-----<View By Categories>-----	
1. Tablets	
2. Mobile Phones	
3. TVs	
4. Exit to Main Menu	
Select a Category:	

Figure 18: View By Categories

This screenshot illustrates the "View By Categories" sub-menu, which I have integrated into my "Hemanth's Techhub Inventory System" to provide a more refined way for users to browse inventory items. I designed this screen to first present a list of available categories: "Tablets," "Mobile Phones," and "TVs," each assigned a clear numerical option. There is also an option to "Exit to Main Menu," ensuring users can easily navigate back to the primary system interface. Below this list, my system prompts the user to "Select a Category," guiding their choice. This screen's core function is to act as an intermediary, allowing users to specify a particular product type before my system displays the filtered inventory, enhancing the user experience by enabling more targeted Browse.



-----<Hemanth's Techhub Inventory System>-----			
-----< Mobile Phones >-----			
ID	Name	Price	Quantity
2	iPhone 16 Pro Max	RM7999	8
5	Google Pixel 9 Pro	RM4799	3
6	Apple Watch Series X	RM1899	7
9	Samsung Galaxy Z Fold 6	RM7899	4

Figure 19: View By Categories Mobile Phones

This screenshot showcases a specific categorized view within my "Hemanth's Techhub Inventory System," accessible after selecting "Mobile Phones" from the "View By Categories" menu. I have meticulously designed this display to present only the items relevant to the chosen category, maintaining the clear, tabular layout seen in the full inventory view. The screen begins with the consistent system header, followed by a specific "Mobile Phones" sub-header that immediately informs the user of the filtered content. Below this, the familiar columns for "ID," "Name," "Price," and "Quantity" are present, ensuring consistent data presentation. Each mobile phone-related item, including accessories like smartwatches that I have categorized similarly, is listed with its corresponding details. As with the full inventory, I maintain the visual cue of highlighting quantities of 5 or less in red to draw attention to low stock levels. After the categorized list is displayed, my system prompts the user to "Press Enter to Return to Menu," seamlessly guiding them back to the category selection screen for further Browse or to the main menu. This specific view's purpose is to allow users to quickly and efficiently review all mobile phone-related inventory without being distracted by other product types.

```
-----<Hemanth's Techhub Inventory System>-----
-----<INVENTORY>-----
ID      Name      Price      Quantity
1      Samsung Galaxy Tab S9      RM4999      4
2      iPhone 16 Pro Max      RM7999      6
3      iPad Air 6th Gen      RM3899      1
4      Sony Bravia XR A95L      RM15999      8
5      Google Pixel 9 Pro      RM4799      3
6      Apple Watch Series X      RM1899      7
7      Samsung Soundbar Q990D      RM3299      2
8      Xiaomi Pad 7      RM1699      5
9      Samsung Galaxy Z Fold 6      RM7899      4

Select an Item to Sell (e to Exit to Main Menu): 1

How many do you want to sell?: 2

Remaining Quantity for Samsung Galaxy Tab S9 :2

Press Enter to Return to Menu
```

Figure 20: Selling an item

```
-----<Hemanth's Techhub Inventory System>-----
-----<INVENTORY>-----
ID      Name      Price      Quantity
1      Samsung Galaxy Tab S9      RM4999      2
2      iPhone 16 Pro Max      RM7999      6
3      iPad Air 6th Gen      RM3899      1
4      Sony Bravia XR A95L      RM15999      8
5      Google Pixel 9 Pro      RM4799      3
6      Apple Watch Series X      RM1899      7
7      Samsung Soundbar Q990D      RM3299      2
8      Xiaomi Pad 7      RM1699      5
9      Samsung Galaxy Z Fold 6      RM7899      4

Select an Item to Sell (e to Exit to Main Menu):
```

Figure 21: Updated quantity for sold item

This is the interface for the "Sell Electronic Devices or Accessories" function within my "Hemanth's Techhub Inventory System," which is accessed via Option 3 from the main menu. I have designed this process to first display the entire inventory, allowing the user to clearly see all available items, their IDs, names, prices, and current quantities, which is crucial for making a selling decision. Below this comprehensive list, my system presents a clear prompt: "Select an Item to Sell (e to Exit to Main Menu):". This prompt not only guides the user to input the ID of the item they wish to sell but also provides a convenient escape option ('e') to return to the main menu at any point. The layout ensures that all necessary information is visible before the user commits to a sale, making the selling process intuitive and error-resistant.

```
-----<Hemanth's Techhub Inventory System>-----
-----<INVENTORY>-----
ID      Name                Price      Quantity
1       Samsung Galaxy Tab S9  RM4999    4
2       iPhone 16 Pro Max      RM7999    8
3       iPad Air 6th Gen       RM3899    5
4       Sony Bravia XR A95L    RM15999   8
5       Google Pixel 9 Pro     RM4799    3
6       Apple Watch Series X    RM1899    7
7       Samsung Soundbar Q990D  RM3299    2
8       Xiaomi Pad 7            RM1699    5
9       Samsung Galaxy Z Fold 6 RM7899    4

Select an Item to Restock (e to Exit to Main Menu): 3

How many do you want to restock?: 2

Remaining Quantity for iPad Air 6th Gen :7

Press Enter to Return to Menu
```

Figure 22: Restocking an item

```
-----<Hemanth's Techhub Inventory System>-----
-----<INVENTORY>-----
ID      Name                Price      Quantity
1       Samsung Galaxy Tab S9  RM4999    4
2       iPhone 16 Pro Max      RM7999    8
3       iPad Air 6th Gen       RM3899    7
4       Sony Bravia XR A95L    RM15999   8
5       Google Pixel 9 Pro     RM4799    3
6       Apple Watch Series X    RM1899    7
7       Samsung Soundbar Q990D  RM3299    2
8       Xiaomi Pad 7            RM1699    5
9       Samsung Galaxy Z Fold 6 RM7899    4

Select an Item to Restock (e to Exit to Main Menu):
```

Figure 23: Updated quantity for restocked item

This displays the "Items Restock" interface within my "Hemanth's Techhub Inventory System," accessible by selecting Option 4 from the main menu. I have designed this screen to first present the user with a comprehensive view of the entire inventory, similar to the selling function, so they can clearly identify which items need replenishment and their current quantities. This visual context is critical for effective restocking decisions. Below the inventory list, my system prompts the user with "Select an Item to Restock (e to Exit to Main Menu):". This prompt directs the user to input the ID of the item they wish to add stock to, while also offering an 'e' option for a convenient return to the main menu if they decide not to proceed with restocking. The clear layout and direct prompting ensure that the user can efficiently manage and update their inventory levels.

```
Exporting inventory to INVENTORY.CSV...  
Inventory exported to INVENTORY.CSV successfully.  
Press Enter to Return to Menu
```

Figure 24: Export to CSV file

This screenshot depicts the process when the "Export Inventory" option (Option 5) is selected from the main menu of my "Hemanth's Techhub Inventory System." I have designed this operation to be straightforward, providing immediate feedback to the user regarding the export status. As soon as the user chooses to export, my system displays a clear message indicating the ongoing process: "Exporting inventory to INVENTORY.CSV..." This informs the user that the system is actively working on saving the inventory data. Upon successful completion of the export, my system then confirms this by stating "Inventory exported to INVENTORY.CSV successfully." This positive feedback assures the user that their data has been saved as intended. Finally, I prompt the user to "Press Enter to Return to Menu," ensuring a smooth transition back to the main functionalities of the system. This screen's sole purpose is to efficiently and transparently handle the saving of the entire inventory database to an external file.

```
-----<Hemanth's Techhub Inventory System>-----  
-----MAIN MENU-----  
1.View Storage Inventory  
2.View Items By ... Category  
3.Sell Electronic Devices or Accessories  
4.Items Restock  
5.Export Inventory  
6.Exit the Program  
Select an Option: 6  
Are you sure you want to exit (y/n): _
```

Figure 25: Exiting the system

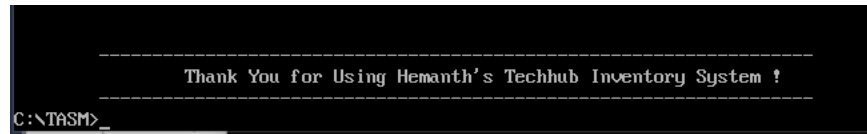


Figure 26: Thank you message upon exiting system

This first screenshot illustrates the "Exit the Program" prompt within my "Hemanth's Techhub Inventory System," which is triggered when Option 6 is selected from the main menu. I have intentionally incorporated a confirmation step to prevent accidental program termination. Upon selection, my system displays a direct question: "Are you sure you want to exit (y/n):", prompting the user to explicitly confirm their intention. This simple 'y' or 'n' input mechanism provides a crucial layer of user control, ensuring that program shutdown is a deliberate action. This screen's function is purely to validate the user's desire to exit before proceeding to close the application.

Following a confirmed exit, the second screenshot shows the "Thank You" message. I have designed this final screen to provide a polite and professional closing for the user's interaction with my system. It displays a clear and prominent "Thank You for Using Hemanth's Techhub Inventory System !" message, framed by a decorative border.

5.0 Source Code with explanation

```
1 |.model small
2 |.stack 1000h ; Increased stack size to 4KB (from 100h)
3 |.386
4 |.data
5 |
6 |; Syntax
7 |CRLF DB 13,10,'$'
8 |
9 |; Validations
10 |InvalidMsg DB 13,10, 'Invalid input. Please try again.', 13,10,10,10,10,'$'
11 |CannotSell DB 13,10, 'Cannot Sell More than the Available Quantity. $'
12 |MaximumStock DB 13,10, 'The Item will Exceed Maximum Stock (9)! $'
13 |
14 |; Strings for Main Menu Display, Exit & UserInput - Store name reverted
15 |; Added option 6 for Export
16 |Mheader DB 13, '-----<Hemanth's Techhub Inventory System>-----'
17 |DB 13, 10, 9, '-----MAIN MENU-----'
18 |DB 13, 10, 10, '1.View Storage Inventory'
19 |DB 13, 10, '2.View Items By ... Category'
20 |DB 13, 10, '3.Sell Electronic Devices or Accessories'
21 |DB 13, 10, '4.Items Restock'
22 |DB 13, 10, '5.Export Inventory' ; SWAPPED: Old was '5.Exit the Program'
23 |DB 13, 10, '6.Exit the Program', 13, 10, '$' ; SWAPPED: Old was '6.Export Inventory'
24 |user_input DB 13,10, 'Select an Option: $'
25 |EnterRet DB 13,10, 'Press Enter to Return to Menus',13,10
26 |Exit DB 13,10,'Are you sure you want to exit (y/n): $',13,10
27 |TyMsg DB 13,10,9,'-----'
28 |DB 13,10,9,9,'Thank You for Using Hemanth's Techhub Inventory System !'
29 |DB 13,10,9,'-----','$'
30 |
31 |; View Inventory Header - Alignment tabs reverted to old code
32 |INVTheader DB 13,10,'-----<Hemanth's Techhub Inventory System>-----'
33 |DB 13,10,9,'-----<INVENTORY>-----'
34 |DB 13,10,'ID',9,'Name',9,9,9,9,'Price',9,9,'Quantity',13,10,'$'
35 |
36 |; Sell Items Strings
37 |SellOption DB 13,10,'Select an Item to Sell (e to Exit to Main Menu): $'
38 |SellQuantity DB 13,10,'How many do you want to sell?: $'
39 |Remain_Qty DB 13,10,'Remaining Quantity for $'
40 |
41 |; Restock Items Strings
42 |RestockQuantity DB 13,10,'How many do you want to restock?: $'
43 |RestockOption DB 13,10,'Select an Item to Restock (e to Exit to Main Menu): $'
44 |
45 |; Categories Strings - Store name reverted and alignment tabs reverted to old code
46 |CatHeader DB 13, '-----<Hemanth's Techhub Inventory System>-----'
47 |DB 13,10,9, '-----<View By Categories>-----'
48 |DB 13,10,'1. Tablets'
49 |DB 13,10,'2. Mobile Phones'
50 |DB 13,10,'3. TVs'
51 |DB 13,10,'4. Exit to Main Menu',13,10,'$'
52 |CatOption DB 13,'Select a Category: $'
53 |TabletHeader DB 13, '-----<Hemanth's Techhub Inventory System>-----'
54 |DB 13,10,9, '-----< Tablets >-----'
55 |DB 13,10,'ID',9,'Name',9,9,9,9,'Price',9,9,'Quantity',13,10,'$'
56 |PhoneHeader DB 13, '-----<Hemanth's Techhub Inventory System>-----'
57 |DB 13,10,9, '-----< Mobile Phones >-----'
58 |DB 13,10,'ID',9,'Name',9,9,9,9,'Price',9,9,'Quantity',13,10,'$'
59 |TVHeader DB 13, '-----<Hemanth's Techhub Inventory System>-----'
60 |DB 13,10,9, '-----< TVs >-----'
61 |DB 13,10,'ID',9,'Name',9,9,9,9,'Price',9,9,'Quantity',13,10,'$'
62 |
63 |; Inventory
64 |item0 DB '1',9,'Samsung Galaxy Tab S9',9,9,'RM4999',9,9,'$' ; Tablet
65 |item1 DB '2',9,'iPhone 16 Pro Max',9,9,'RM7999',9,9,'$' ; Mobile Phone
66 |item2 DB '3',9,'iPad Air 6th Gen',9,9,'RM3899',9,9,'$' ; Tablet
67 |item3 DB '4',9,'Sony Bravia XR A95L',9,9,'RM15999',9,9,'$' ; TV
68 |item4 DB '5',9,'Google Pixel 9 Pro',9,9,'RM4799',9,9,'$' ; Mobile Phone
69 |item5 DB '6',9,'Apple Watch Series X',9,9,'RM1899',9,9,'$' ; Mobile Phone accessory (Smartwatch)
70 |item6 DB '7',9,'Samsung Soundbar Q990D',9,9,'RM3299',9,9,'$' ; TV accessory (Soundbar)
71 |item7 DB '8',9,'Xiaomi Pad 7',9,9,9,'RM1699',9,9,'$' ; Tablet (Mini Tablet)
72 |item8 DB '9',9,'Samsung Galaxy Z Fold 6',9,9,9,'RM7899',9,9,'$' ; Mobile Phone (Fold Phone)
73 |
```

Figure 27: Source Code Screenshot 1

At the very top of my assembly program, I establish the fundamental configuration. The `.model` small directive is how I define the memory model, indicating that my program will operate with single segments for both code and data, a structure I find efficient for the scope of this inventory system. Directly following that, `.stack 1000h` allocates a generous 4KB for the program's stack, which I deemed necessary to accommodate various procedure calls and local variables without overflow issues. The `.386` directive is then included to inform the assembler that I intend to use instructions specific to the Intel 80386 processor or newer, allowing me to leverage more advanced capabilities.

Moving into the `.data` segment, this is the dedicated area where I declare and initialize all the static data and messages that my program will utilize. For basic formatting, I define `CRLF` for reliable newlines. I have also meticulously crafted various validation messages like `InvalidMsg` for incorrect user input, `CannotSell` to prevent over-selling, and `MaximumStock` to ensure inventory levels don't exceed their limits. For my user interface, I store all the necessary strings for the main menu, including `Mheader` which prominently displays "Hemanth's Techhub Inventory System" and the menu options, `user_input` to prompt for selection, `EnterRet` for returning to the menu, and `Exit` and `TyMsg` for the program's graceful conclusion. The specific headers for viewing the entire inventory (`INVTheader`), and for the dedicated sell (`SellOption`, `SellQuantity`, `Remain_Qty`) and restock (`RestockQuantity`, `RestockOption`) functionalities are all precisely defined here. Furthermore, for my categorized views, I include `CatHeader` and distinct headers for each product type like `TabletHeader`, `PhoneHeader`, and `TVHeader`. Finally, this section contains the detailed definitions of all my inventory items, from `item0` to `item8`, each clearly specifying its ID, product name, and price, providing the static display data for my inventory lists.

```

;Items Name for display (used in Sell/Restock confirmation)
itemn0 DB 'Samsung Galaxy Tab S9 :$'
itemn1 DB 'iPhone 16 Pro Max :$'
itemn2 DB 'iPad Air 6th Gen :$'
itemn3 DB 'Sony Bravia XR A95L :$'
itemn4 DB 'Google Pixel 9 Pro :$'
itemn5 DB 'Apple Watch Series X :$'
itemn6 DB 'Samsung Soundbar Q990D :$'
itemn7 DB 'Xiaomi Pad 7 :$'
itemn8 DB 'Samsung Galaxy Z Fold 6 :$'

; Items' Quantity
q_Item0 DB 4
q_Item1 DB 6
q_Item2 DB 1
q_Item3 DB 8
q_Item4 DB 3
q_Item5 DB 7
q_Item6 DB 2
q_Item7 DB 5
q_Item8 DB 4

; --- New for Export Feature ---
export_filename DB 'INVENTORY.CSV',0 ; Filename for export, null-terminated
export_header DB 'ID,Name,Price,Quantity',13,10,'$' ; CSV header line
export_success DB 13,10,'Inventory exported to INVENTORY.CSV successfully.$'
export_fail DB 13,10,'Error exporting inventory.$'

; Explicitly define the export message as a variable
export_processing_msg DB 'Exporting inventory to INVENTORY.CSV...$'

file_handle DW ? ; Variable to store the file handle

; Separate strings for item names and prices for easier CSV export
itemn0_export DB 'Samsung Galaxy Tab S9$'
itemp0_export DB 'RM4999$'
itemn1_export DB 'iPhone 16 Pro Max$'
itemp1_export DB 'RM7999$'
itemn2_export DB 'iPad Air 6th Gen$'
itemp2_export DB 'RM3899$'
itemn3_export DB 'Sony Bravia XR A95L$'
itemp3_export DB 'RM15999$'
itemn4_export DB 'Google Pixel 9 Pro$'
itemp4_export DB 'RM4799$'
itemn5_export DB 'Apple Watch Series X$'
itemp5_export DB 'RM1899$'
itemn6_export DB 'Samsung Soundbar Q990D$'
itemp6_export DB 'RM3299$'
itemn7_export DB 'Xiaomi Pad 7$'
itemp7_export DB 'RM1699$'
itemn8_export DB 'Samsung Galaxy Z Fold 6$'
itemp8_export DB 'RM7899$'

```

Figure 28: Source Code Screenshot 2

Continuing with the explanation of my system's data definitions for your documentation, this section focuses on the specific strings and variables I use to manage and export inventory data. I have defined itemn0 through itemn8 as dedicated strings containing the full names of each product, immediately followed by a colon, which I specifically use for clear and direct display during sell and restock confirmation prompts. Paired with these are q_Item0 through q_Item8, which are byte variables that I use to store the dynamic, current quantity of each corresponding inventory item; these are central to tracking stock levels. Further down, for my robust export feature, I declare export_filename as the name of the CSV file where the inventory will be saved, and export_header provides the necessary column titles for that file. I also include export_success and export_fail messages to clearly communicate the outcome of the export operation to the user, with

export_processing_msg serving as an intermediate status update. The file_handle variable is reserved to store the unique identifier my operating system provides when the file is opened for writing. To ensure proper formatting for the CSV, I have created separate itemnX_export and itempX_export strings that hold the item names and prices without the display-specific formatting, making them ready for direct export.

```
; Tables of addresses for item names and prices for easy iteration
item_names_table DW item0_export, itemn1_export, itemn2_export, itemn3_export, itemn4_export, itemn5_export, itemn6_export, itemn7_export, itemn8_export
item_prices_table DW itemp0_export, itemp1_export, itemp2_export, itemp3_export, itemp4_export, itemp5_export, itemp6_export, itemp7_export, itemp8_export

; Temporary buffer for single character writes to file (Fix for Illegal indexing mode)
temp_char_buffer DB ?, '$'
; --- End-New for Export Feature ---

; --- Authentication Variables ---
; Username
correct_username_str DB 'Hemanth',0 ; The correct username, null-terminated
correct_username_len EQU ($ - correct_username_str - 1) ; Length of the actual username (7)
username_prompt DB 13,10,'Enter Username: $'
no_user_found_msg DB 13,10,'No user found. Exiting.$'
username_input_buffer DB 15 ; Max characters for username
; Actual number of characters read
entered_username DB 15 DUP (?) ; Buffer to store the entered username string

; Password
correct_password_str DB 'Hemanth1234',0 ; The actual password, null-terminated
correct_password_len EQU ($ - correct_password_str - 1) ; Length of the actual password (11)
password_prompt DB 13,10,'Password for Hemanth's Techhub Inventory System : $'
incorrect_password_msg DB 13,10,'Incorrect Password. Exiting.$'

; Buffer for password input using INT 21h, AH=0Ah
password_input_buffer DB 15 ; Max characters for password
; Actual number of characters read
entered_password DB 15 DUP (?) ; Buffer to store the entered password string (will be null-terminated)
; --- End Authentication Variables ---

.code

;Display Messages
DisplayMsg Macro Msg
    lea dx, Msg ; load message address
    mov ah, 09h ; function to display string
    int 21h ; call DOS
EndM

;Display Items and Quantity
DisplayQty Macro Qty, Color
    LOCAL LessThanFiveQty, DisplayNormal, EndQty

    mov dl, Qty
    cmp dl, 5 ; Compare the quantity with 5
    jl LessThanFiveQty ; Jump to LessThanFiveQty if quantity is less than 5
    jge DisplayNormal

    DisplayNormal:
        ; Display the quantity normally
        mov dl, Qty ; Move the quantity back to DL
        add dl, '0' ; Convert numerical value to ASCII character
        mov ah, 02h ; Display character
        int 21h ; Call DOS
        jmp EndQty ; Jump to the end of the macro

    LessThanFiveQty:
        ; Display the quantity in red color
        mov al, Qty ; Move the quantity to AL
        add al, '0' ; Convert numerical value to character
        mov ah, 09h ; Display character
        mov bh, 0 ; Page number (0 for current page)
        mov bl, Color ; Text attribute (red color)
        mov cx, 1 ; Set to print only 1 char
        int 10h ; Call BIOS
        jmp EndQty

    EndQty:
EndM
```

Figure 29: Source Code Screenshot 3

Furthermore, this next section of my .data segment is crucial for the efficient handling of inventory information, especially for the export functionality, and for securing access to my system. I have defined item_names_table and item_prices_table as arrays of addresses. This clever design allows me to iterate through all my product names and prices systematically when I need to write them to the CSV file, rather than accessing each item individually, which simplifies my export logic. The temp_char_buffer is a small but essential temporary storage area I created to facilitate writing

single characters to the file during the export process, specifically to resolve an "Illegal indexing mode" issue I encountered.

Transitioning into the .code segment, I begin by defining my foundational macros, which are essentially reusable blocks of code I frequently employ. The DisplayMsg macro is a core utility I created to easily print any dollar-terminated string to the console; by simply calling this macro with the message's address, I can display information to the user without rewriting the same DOS interrupt calls. Similarly, the DisplayQty macro is another key component I designed to intelligently display item quantities. This macro not only converts the numerical quantity into an ASCII character for display but also includes a built-in visual cue: if the quantity is 5 or less, I specifically instruct it to display the number in red, immediately alerting the user to low stock levels on my inventory screens, enhancing the system's usability.

```

; Selling an item
SellItem MACRO item,quantityitem
    LOCAL CantSell
    DisplayMsg SellQuantity
    mov ah, 01h
    int 21h

    sub al, '0'
    cmp [quantityitem],al ; FIX: Dereference quantityitem
    jl CantSell ; if input > quantity, cant sell

    sub [quantityitem], al ; FIX: Dereference quantityitem ; Subtract the quantity from the corresponding item

    ;Display the remain quantity
    DisplayMsg CRLF
    DisplayMsg Remain_Qty
    DisplayMsg item
    mov dl, [quantityitem] ; FIX: Dereference quantityitem
    DisplayQty dl, 4

    DisplayMsg CRLF
    call ReturnToMenu
    jmp sell_items
ENDM

CantSell:
    DisplayMsg CRLF
    DisplayMsg CannotSell
    DisplayMsg CRLF
    call ReturnToMenu
    jmp sell_items ; FIX: Changed call to jmp for looping
ENDM

;Restocking an item
RestockItem MACRO item_name_msg, quantity_var_address
    LOCAL CheckQuantity, RestockContinue, RestockExitMacro

    ; --- DIAGNOSTIC PRINT: Check if execution reaches inside the macro ---
    mov al, 'X'
    mov ah, 02h
    int 21h ; This should print 'X' to the console.
    ; -----

    DisplayMsg RestockQuantity
    mov ah, 01h
    int 21h

    sub al, '0' ; AL now holds the numerical quantity to restock (e.g., 2)

    ; Get current quantity VALUE and calculate potential new quantity
    mov bl, [quantity_var_address] ; FIX: Load current quantity VALUE from memory into BL
    add bl, al ; Add input quantity (AL) to current quantity (BL) -> sum in BL

    cmp bl, 9 ; Compare the potential new quantity (in BL) with max stock (9)
    jg CheckQuantity ; If sum > 9, jump to error message

    ; If not exceeding max stock, update the quantity
    mov [quantity_var_address], bl ; FIX: Store the new sum (in BL) back into memory

RestockContinue: ; Label for normal flow after quantity update
    DisplayMsg CRLF
    DisplayMsg Remain_Qty
    DisplayMsg item_name_msg ; Use the item's name string for display
    mov dl, [quantity_var_address] ; FIX: Display the UPDATED quantity VALUE from memory
    DisplayQty dl, 4

    DisplayMsg CRLF
    call ReturnToMenu
    jmp RestockExitMacro ; Jump to exit the macro's logic

CheckQuantity: ; Label for exceeding max stock
    DisplayMsg CRLF
    DisplayMsg MaximumStock
    DisplayMsg CRLF
    call ReturnToMenu
    jmp RestockExitMacro ; Jump to exit the macro's logic

RestockExitMacro: ; End of macro's internal flow, allows control to pass back
ENDM

```

Figure 30: Source Code Screenshot 4

This segment of my code contains two crucial macros that drive the core inventory management actions: SellItem and RestockItem.

The SellItem macro is how I manage the process of deducting stock from my inventory. When invoked, I first use DisplayMsg SellQuantity to ask the user how many units they wish to sell. After reading their input, I immediately convert it to a numerical value and then perform a critical check: I compare the requested quantity against the currently available quantityitem. If the user attempts to sell more than what is in stock, I direct the program to CantSell, where I display a

CannotSell message, preventing negative inventory. However, if the quantity is valid, I proceed to subtract the sold amount from quantityitem and then display the Remain_Qty using the DisplayQty macro, which even highlights low stock. Finally, I ensure the user can return to the main menu.

Similarly, the RestockItem macro is my robust solution for adding items back into stock. Upon activation, I first prompt the user for the quantity they wish to restock using DisplayMsg RestockQuantity. After capturing their input, I calculate the potential new stock level by adding the restock quantity to the item's current quantity. A vital part of my design here is the CheckQuantity logic: I rigorously compare this potential new quantity against a maximum stock limit of 9. If adding the requested items would cause the stock to exceed this limit, I display a MaximumStock warning, preventing over-stocking. If the restock is valid, I update the quantity_var_address with the new total and then clearly display the Remain_Qty. (You might also notice a small mov al, 'X' diagnostic print I temporarily added in its early development stages to confirm the macro's execution during testing!) This careful design ensures that my inventory remains accurate and within acceptable limits.

```
;Main Function
MAIN PROC
    mov ax, @Data      ; Set data segment
    mov ds, ax         ; Data register

    ; --- Username Protection Start ---
    call clear_screen
    DisplayMsg username_prompt

    ; Read username using buffered input (INT 21h, AH=0Ah)
    mov ah, 0Ah        ; Function: Buffered input
    lea dx, username_input_buffer ; DX points to the buffer structure
    int 21h            ; Call DOS

    ; Get the actual length of the entered username
    mov bl, [username_input_buffer + 1] ; Actual length (excluding CR/LF) into BL
    mov bh, 0 ; Clear BH (makes BX = length for CX later)

    ; Copy the entered username from the input buffer to our 'entered_username' variable
    mov si, offset username_input_buffer + 2 ; Source: actual entered characters
    mov di, offset entered_username          ; Destination: our dedicated buffer
    mov cx, bx ; CX = actual length for REP MOVSB

    rep movsb ; Copy string byte by byte

    ; Add null terminator to the copied entered_username string
    mov byte ptr [di], 0 ; DI is already at the end of the copied string

    ; Compare entered username with correct_username_str
    ; First, compare lengths
    mov cl, correct_username_len ; Length of correct username (into CL)
    cmp bl, cl ; Compare lengths (BL vs CL)
    jne incorrect_username_exit ; If lengths don't match, it's incorrect

    ; If lengths match, compare content byte by byte
    mov si, offset correct_username_str ; Source: correct username string
    mov di, offset entered_username    ; Destination: entered username string
    mov cx, bx ; CX = length for comparison (restore actual length into CX)
    repe cmpsb ; Compare strings (repeating while bytes are equal)
    jne incorrect_username_exit ; If not equal (ZF=0), username is incorrect

    ; Username is correct, proceed to password check
    jmp username_correct
```

Figure 31: Source Code Screenshot 5

Upon program execution, the very first task I perform is to initialize the data segment, achieved by moving the address of my .data segment into the DS register. This crucial setup ensures that my program can correctly access all the variables and strings I have defined. Immediately following this, I initiate my system's security protocol by clearing the screen and presenting the username_prompt to the user, requesting their credentials. To capture the username efficiently, I employ DOS's buffered input function (INT 21h, AH=0Ah), which reads the user's input into my username_input_buffer and also conveniently stores its actual length. After the input, I meticulously extract this length and then copy the entered characters into my dedicated entered_username variable, ensuring it's properly null-terminated. My authentication logic then proceeds with a rigorous two-step validation: I first compare the length of the entered_username against my predefined correct_username_len. If these lengths do not match, I know it's an invalid attempt and divert execution to incorrect_username_exit. However, if the lengths align, I then perform a precise character-by-character comparison of the entered_username against the correct_username_str. Only if both these stringent checks pass successfully does my system jmp username_correct, allowing the user to advance to the next stage of authentication for the "Hemanth's Techhub Inventory System."

```

incorrect_username_exit:
    DisplayMsg CRLF
    DisplayMsg no_user_found_msg
    call exit_main ; Exit program
    ; --- Username Protection End ---

username_correct:
    ; --- Password Protection Start ---
    call clear_screen
    DisplayMsg password_prompt

    ; Read password using buffered input (INT 21h, AH=0Ah)
    mov ah, 0Ah ; Function: Buffered input
    lea dx, password_input_buffer ; DX points to the buffer structure
    int 21h ; Call DOS

    ; Get the actual length of the entered password
    mov bl, [password_input_buffer + 1] ; Actual length (excluding CR/LF) into BL
    mov bh, 0 ; Clear BH for BX to be 16-bit

    ; Copy the entered password from the input buffer to our 'entered_password' variable
    mov si, offset password_input_buffer + 2 ; Source: actual entered characters
    mov di, offset entered_password ; Destination: our dedicated buffer
    mov cx, bx ; CX = actual length for REP MOVSB

    rep movsb ; Copy string byte by byte

    ; Add null terminator to the copied entered_password string
    mov byte ptr [di], 0 ; DI is already at the end of the copied string

    ; Compare entered password with correct_password_str
    ; First, compare lengths
    mov cl, correct_password_len ; Length of correct password (into CL)
    cmp bl, cl ; Compare lengths (BL vs CL)
    jne incorrect_password_exit ; If lengths don't match, it's incorrect

    ; If lengths match, compare content byte by byte
    mov si, offset correct_password_str ; Source: correct password string
    mov di, offset entered_password ; Destination: entered password string
    mov cx, bx ; CX = length for comparison (restore actual length into CX)
    repe cmpsb ; Compare strings (repeating while bytes are equal)
    jne incorrect_password_exit ; If not equal (ZF=0), password is incorrect

    ; Password is correct, proceed to main loop
    jmp main_loop

incorrect_password_exit:
    DisplayMsg CRLF
    DisplayMsg incorrect_password_msg
    call exit_main ; Exit program
    ; --- Password Protection End ---

main_loop:

    call display_menu
    mov ah, 01h ; Read user character input
    int 21h

    ; Compare user_input and direct to selected option
    cmp al, '1'
    je inventory_menu
    cmp al, '2'
    je category_menu
    cmp al, '3'
    je sell_items
    cmp al, '4'
    je restock_items
    cmp al, '5'
    je export_inventory ; SWAPPED: Old was '5.exit_confirmed'
    cmp al, '6'
    je exit_confirmed ; SWAPPED: Old was '6.export_inventory'
    ; Removed direct jumps for 7,8,9 as main menu only goes to 6 options now.
    ; Sell/Restock functions will still handle 1-9.

    ; Invalid Input
    DisplayMsg CRLF
    DisplayMsg InvalidMsg
    DisplayMsg CRLF
    jmp main_loop

```

Figure 32: Source Code Screenshot 6

When an incorrect username is entered, my system's incorrect_username_exit block is activated. Here, I promptly display the no_user_found_msg to inform the user that their credentials are not

recognized, and then I initiate call `exit_main` to gracefully terminate the program, ensuring system security.

Upon a successful username validation, my program jumps to `username_correct`, which marks the beginning of the "Password Protection" phase. Similar to username verification, I first clear the screen and then present the `password_prompt` to the user, asking for their password. I read the password input using the same buffered `INT 21h, AH=0Ah` method, storing it in my `password_input_buffer`. I then extract its actual length and copy the characters into my `entered_password` variable. The validation process mirrors the username check: I first compare the length of the `entered_password` with my `correct_password_len`. If the lengths mismatch, or if a byte-by-byte comparison against `correct_password_str` reveals any discrepancy, I consider it an `incorrect_password_exit` and, just like with an invalid username, I display `incorrect_password_msg` and exit the program. Only if both the length and content of the password are precisely correct does my system `jmp main_loop`, granting access to the core functionalities.

The `main_loop` is the heart of my inventory system's interactivity, where I continuously present the user with options. Inside this loop, I first call `display_menu` to render the main menu screen. Then, I wait for the user to select an option by reading a single character input. Based on their input, I use a series of conditional jumps (`cmp al, '1', je inventory_menu`, and so on) to direct the program's flow to the corresponding functionality, such as viewing inventory, accessing categories, or initiating sell, restock, and export operations. I have also configured the options to handle the program exit (`exit_confirmed`). If the user enters an unrecognized option, I display an `InvalidMsg` and then loop back to `main_loop`, prompting them again until a valid selection is made, ensuring a robust user experience.

```

;Loops
;Display Main Menu and UserInput String
display_menu:
    call clear_screen ; Clear screen before displaying menu
    DisplayMsg Mheader
    DisplayMsg user_input
    ret

;Option 2: View Inventory
inventory_menu:
    call view_inventory
    call ReturnToMenu
    jmp main_loop

view_inventory:
    call clear_screen ; Clear screen before displaying inventory
    DisplayMsg INVTheader

    ; Display All Items
    DisplayMsg item0
    DisplayQty q_Item0, 4
    DisplayMsg CRLF

    DisplayMsg item1
    DisplayQty q_Item1, 4
    DisplayMsg CRLF

    DisplayMsg item2
    DisplayQty q_Item2, 4
    DisplayMsg CRLF

    DisplayMsg item3
    DisplayQty q_Item3, 4
    DisplayMsg CRLF

    DisplayMsg item4
    DisplayQty q_Item4, 4
    DisplayMsg CRLF

    DisplayMsg item5
    DisplayQty q_Item5, 4
    DisplayMsg CRLF

    DisplayMsg item6
    DisplayQty q_Item6, 4
    DisplayMsg CRLF

    DisplayMsg item7
    DisplayQty q_Item7, 4
    DisplayMsg CRLF

    DisplayMsg item8
    DisplayQty q_Item8, 4
    DisplayMsg CRLF
    ret

```

Figure 33: Source Code Screenshot 7

The `display_menu` procedure is a fundamental routine that I call whenever the main menu needs to be presented to the user. Its function is straightforward: I first ensure a clean slate by calling `clear_screen`, then I display the comprehensive `Mheader` that includes the system title and all menu options, followed by the `user_input` prompt, guiding the user to make a selection. Once these elements are shown, the procedure simply returns, allowing the `main_loop` to capture the user's choice.

The `inventory_menu` procedure serves as the dedicated entry point when the user opts to view the entire storage inventory. From here, I initiate the `view_inventory` procedure to handle the actual display of all items. After the inventory list has been presented, I call `ReturnToMenu`, which pauses the program until the user presses Enter, providing them ample time to review the data. Finally, I `jmp main_loop` to seamlessly transition the user back to the main menu for their next action.

The `view_inventory` procedure is where the entire contents of my inventory are laid out for the user. I begin by clearing the screen to provide an uncluttered view, and then I display the `INVTheader` which sets up the columns for ID, Name, Price, and Quantity. Following this, I systematically list each of my nine inventory items (`item0` through `item8`). For each item, I use `DisplayMsg` to print its ID, name, and price details, and then crucially, I use my `DisplayQty` macro to show its current stock quantity, ensuring low stock levels are visually highlighted in red. Each item entry is followed by a CRLF to format it neatly on its own line, creating the readable tabular format I designed. Once all items are displayed, the procedure returns, passing control back to `inventory_menu`.

```

;Sell Menu
sell_items:

    DisplayMsg CRLF
    call view_inventory

    ; Read user's input for item selection
    DisplayMsg SellOption
    mov ah, 01h          ; Read user character input
    int 21h
    cmp al, '1'
    je sellitem0
    cmp al, '2'
    je sellitem1
    cmp al, '3'
    je sellitem2
    cmp al, '4'
    je sellitem3
    cmp al, '5'
    je sellitem4
    cmp al, '6'
    je sellitem5
    cmp al, '7'
    je sellitem6
    cmp al, '8'
    je sellitem7
    cmp al, '9'
    je sellitem8
    cmp al, 'e'
    je main_loop
    cmp al, 'E'
    je main_loop

    ;Invalid Input
    DisplayMsg CRLF
    DisplayMsg InvalidMsg
    DisplayMsg CRLF
    jmp sell_items

sellitem0:
    DisplayMsg CRLF
    SellItem itemn0,q_Item0
    jmp sell_items ; Ensure a jump back after macro
sellitem1:
    DisplayMsg CRLF
    SellItem itemn1,q_Item1
    jmp sell_items ; Ensure a jump back after macro
sellitem2:
    DisplayMsg CRLF
    SellItem itemn2,q_Item2
    jmp sell_items ; Ensure a jump back after macro
sellitem3:
    DisplayMsg CRLF
    SellItem itemn3,q_Item3
    jmp sell_items ; Ensure a jump back after macro
sellitem4:
    DisplayMsg CRLF
    SellItem itemn4,q_Item4
    jmp sell_items ; Ensure a jump back after macro
sellitem5: ; New item sell handler
    DisplayMsg CRLF
    SellItem itemn5,q_Item5
    jmp sell_items ; Ensure a jump back after macro
sellitem6: ; New item sell handler
    DisplayMsg CRLF
    SellItem itemn6,q_Item6
    jmp sell_items ; Ensure a jump back after macro
sellitem7:
    DisplayMsg CRLF
    SellItem itemn7,q_Item7
    jmp sell_items ; Ensure a jump back after macro
sellitem8:
    DisplayMsg CRLF
    SellItem itemn8,q_Item8
    jmp sell_items ; Ensure a jump back after macro

```

Figure 34: Source Code Screenshot 8

This segment of my code defines the `sell_items` procedure, which is the dedicated module I designed to facilitate the selling of inventory items within "Hemanth's Techhub Inventory System." When a user chooses this option, I first ensure they have a complete overview by calling `view_inventory`, displaying the current stock list with all item details. After presenting the inventory, I prompt the user using my `SellOption` message, clearly instructing them to select an item by its ID or to type 'e' to return to the main menu.

Once the user provides their input, I immediately evaluate their choice. A series of comparison and jump instructions (`cmp al, '1', je sellitem0`, and so on) meticulously directs the program flow to the specific `sellitemX` block corresponding to the item ID they entered. If the user decides to exit the selling process, I have accounted for both 'e' and 'E' inputs, which will then `jmp main_loop`, gracefully returning them to the system's central hub. In cases where an invalid input is provided, I display my `InvalidMsg` and then `jmp sell_items`, allowing them to try again without leaving the selling interface.

Each `sellitemX` block, such as `sellitem0` or `sellitem1`, is a simple but powerful component. Within these blocks, I first display a newline for formatting and then directly invoke my `SellItem` macro, passing it two crucial arguments: the item's display name (e.g., `itemn0`) for confirmation messages, and the address of its corresponding quantity variable (e.g., `q_Item0`) so the macro can directly update the stock. After the macro completes its selling logic, I include a `jmp sell_items` to ensure that the program returns to the main selling interface, allowing the user to sell multiple items consecutively without having to re-navigate from the main menu.

```

;Restock Menu
restock_items:
    DisplayMsg CRLF
    call view_inventory

    ; Read user's input for item selection
    DisplayMsg RestockOption
    mov ah, 01h
    int 21h
    cmp al, '1'
    je restock0
    cmp al, '2'
    je restock1
    cmp al, '3'
    je restock2
    cmp al, '4'
    je restock3
    cmp al, '5'
    je restock4
    cmp al, '6'
    je restock5
    cmp al, '7'
    je restock6
    cmp al, '8'
    je restock7
    cmp al, '9'
    je restock8
    cmp al, 'e'
    je main_loop
    cmp al, 'E'
    je main_loop

    ;Invalid Input
    DisplayMsg CRLF
    DisplayMsg InvalidMsg
    DisplayMsg CRLF
    jmp restock_items

restock0:
    DisplayMsg CRLF
    RestockItem itemn0, q_Item0
    jmp restock_items ; Added for consistency
restock1:
    DisplayMsg CRLF
    RestockItem itemn1, q_Item1
    jmp restock_items ; Added for consistency
restock2:
    DisplayMsg CRLF
    RestockItem itemn2, q_Item2
    jmp restock_items ; Added for consistency
restock3:
    DisplayMsg CRLF
    RestockItem itemn3, q_Item3
    jmp restock_items ; Added for consistency
restock4:
    DisplayMsg CRLF
    RestockItem itemn4, q_Item4
    jmp restock_items ; Added for consistency
restock5: ; New item restock handler
    DisplayMsg CRLF
    RestockItem itemn5, q_Item5
    jmp restock_items ; Added for consistency
restock6:
    DisplayMsg CRLF
    RestockItem itemn6, q_Item6
    jmp restock_items ; Added for consistency
restock7:
    DisplayMsg CRLF
    RestockItem itemn7, q_Item7
    jmp restock_items ; Added for consistency
restock8:
    DisplayMsg CRLF
    RestockItem itemn8, q_Item8
    jmp restock_items ; Added for consistency

```

Figure 35: Source Code Screenshot 9

This section of my code defines the `restock_items` procedure, which is the module I designed to facilitate the process of replenishing inventory items in "Hemanth's Techhub Inventory System." When the user selects this option, I first ensure they have a comprehensive view of the current stock by calling `view_inventory`, allowing them to see what needs replenishing. Following this, I present the `RestockOption` prompt, instructing them to select an item by its ID or to type 'e' if they wish to return to the main menu.

Upon receiving the user's input, I meticulously compare their selection using a series of `cmp` and `je` instructions. This logic directs the program flow to the specific `restockX` block corresponding to the item ID they entered, from '1' to '9'. If the user inputs 'e' or 'E', I ensure a smooth return to the `main_loop` of my system. Should an invalid character be entered, I display the `InvalidMsg` and then `jmp restock_items`, gracefully returning them to the restock menu to make a valid selection.

Each individual `restockX` block, such as `restock0` or `restock1`, is built to handle the restock action for a specific item. Within these blocks, I display a newline for clarity and then invoke my `RestockItem` macro. I pass this macro the item's display name (e.g., `itemn0`) for consistent user feedback and, crucially, the address of its corresponding quantity variable (e.g., `q_Item0`), enabling the macro to directly update the stock count. After the `RestockItem` macro completes its operation, I include a `jmp restock_items` to ensure the user is returned to the restock selection interface, allowing them to conveniently restock multiple items without needing to re-navigate from the main menu each time.

```

;Sort By Category Menu
category_menu:
    call clear_screen ; Clear screen before displaying category menu
    DisplayMsg CatHeader
    DisplayMsg CatOption
    mov ah,01h
    int 21h
    cmp al,'1'
    je tablets
    cmp al,'2'
    je phones
    cmp al,'3'
    je tvs
    cmp al,'4'
    je main_loop

    ;Invalid Input
    DisplayMsg CRLF
    DisplayMsg InvalidMsg
    DisplayMsg CRLF
    jmp category_menu

tablets:
    call clear_screen ; Clear screen before displaying tablets
    DisplayMsg TabletHeader
    DisplayMsg CRLF
    DisplayMsg item0
    DisplayQty q_Item0, 4
    DisplayMsg CRLF

    DisplayMsg item2
    DisplayQty q_Item2, 4
    DisplayMsg CRLF

    DisplayMsg item7
    DisplayQty q_Item7, 4
    DisplayMsg CRLF
    call ReturnToMenu
    jmp category_menu

phones:
    call clear_screen ; Clear screen before displaying phones
    DisplayMsg PhoneHeader
    DisplayMsg CRLF
    DisplayMsg item1
    DisplayQty q_Item1, 4
    DisplayMsg CRLF

    DisplayMsg item4
    DisplayQty q_Item4, 4
    DisplayMsg CRLF

    DisplayMsg item5
    DisplayQty q_Item5, 4
    DisplayMsg CRLF

    DisplayMsg item8
    DisplayQty q_Item8, 4
    DisplayMsg CRLF
    call ReturnToMenu
    jmp category_menu

tvs:
    call clear_screen ; Clear screen before displaying TVs
    DisplayMsg TVHeader
    DisplayMsg CRLF
    DisplayMsg item3
    DisplayQty q_Item3, 4
    DisplayMsg CRLF

    DisplayMsg item6
    DisplayQty q_Item6, 4
    DisplayMsg CRLF
    call ReturnToMenu
    jmp category_menu

```

Figure 36: Source Code Screenshot 10

Moving on, the `category_menu` procedure is the entry point for this functionality. When accessed, I first clear the screen to ensure a clean display and then present the `CatHeader`, which lists the available categories: Tablets, Mobile Phones, and TVs, along with an option to exit back to the main menu. I then prompt the user with `CatOption` to select their desired category. Upon receiving their input, I use a series of conditional jumps (`cmp al, '1', je tablets, etc.`) to direct the program to the specific procedure responsible for displaying items within that chosen category. If the user selects '4', I ensure they are returned directly to the `main_loop`. Should an invalid input be provided, I display the `InvalidMsg` and then `jmp category_menu`, prompting them to make a valid selection.

Each specific category procedure, like `tablets`, `phones`, and `tv`s, is meticulously designed to display only the relevant items. For instance, the `tablets` procedure begins by clearing the screen and then displaying `TabletHeader`. Following this, I individually list each item that I have categorized as a tablet (e.g., `item0`, `item2`, `item7`), using `DisplayMsg` for the item details and `DisplayQty` for their current stock levels, just as in the full inventory view. The `phones` and `tv`s procedures follow this same meticulous pattern, displaying only their respective categorized items. After presenting the filtered inventory, I call `ReturnToMenu` to allow the user to review the data at their leisure, and then I `jmp category_menu`, enabling them to easily select another category or return to the main menu for further actions within my "Hemanth's Techhub Inventory System."

```

; --- Export Inventory Procedure ---
export_inventory PROC
    call clear_screen
    DisplayMsg CRLF
    DisplayMsg export_processing_msg
    DisplayMsg CRLF

    ; Create file (INT 21h, AH=3Ch)
    mov ah, 3Ch
    xor cx, cx ; Normal file attributes
    lea dx, export_filename ; Address of filename
    int 21h
    jc export_error_exit_handling ; If CF is set, error creating file (e.g., disk full, invalid path)

    mov [file_handle], ax ; Save returned file handle in AX

    ; Write header to file (INT 21h, AH=40h)
    mov bx, [file_handle] ; File handle in BX
    lea dx, export_header ; Address of header string
    mov cx, 23 ; Length of 'ID,Name,Price,Quantity\r\n'
    mov ah, 40h ; Write to file
    int 21h
    jc export_error_exit_handling ; Check for write error

    ; Loop through each item (0 to 8) to write its data
    mov si, 0 ; Loop counter/index (0-based) for both byte and word tables
    mov cx, 9 ; Number of items to process

export_loop:
    ; Get item ID (loop index + 1) and write
    mov ax, si ; FIX: Copy SI (16-bit) to AX (16-bit). AL now holds the lower byte of SI.
    add al, '1' ; Convert 0-8 to '1'-'9' ASCII
    call write_char_to_file_internal ; Helper to write char to file handle

    ; Write comma
    mov al, ','
    call write_char_to_file_internal

    ; Get item Name from item names table and write
    push si ; Save SI (loop counter)
    mov di, si ; Copy loop counter to DI
    shl di, 1 ; Scale DI by 2 (for word array: 0, 2, 4...)

    lea bx, item_names_table ; BX gets the base address of the table
    mov dx, WORD_PTR [bx + di] ; Get the WORD (address) from item_names_table[bx+di] into DX
    pop si ; Restore SI
    call write_string_to_file_internal

    ; Write comma
    mov al, ','
    call write_char_to_file_internal

    ; Get item Price from item prices table and write
    push si ; Save SI (loop counter)
    mov di, si ; Copy loop counter to DI
    shl di, 1 ; Scale DI by 2

    lea bx, item_prices_table ; BX gets the base address of the table
    mov dx, WORD_PTR [bx + di] ; Get the WORD (address) from item_prices_table[bx+di] into DX
    pop si ; Restore SI
    call write_string_to_file_internal

    ; Write comma
    mov al, ','
    call write_char_to_file_internal

    ; Get quantity from q_ItemX (direct access using si index) and write
    mov bl, [q_Item0 + si] ; q_Item0 is a byte array, si is 0, 1, ..., 8. This is correct.
    call write_byte_as_ascii_to_file_internal ; Helper to convert byte to ASCII and write

    ; Write CRLF
    mov al, 13 ; Carriage Return
    call write_char_to_file_internal
    mov al, 10 ; Line Feed
    call write_char_to_file_internal

    inc si ; Move to next item index (0-8)
    loop export_loop ; Decrement CX and loop if CX > 0

```

Figure 37: Source Code Screenshot 11

This significant section of my code outlines the `export_inventory` procedure, a crucial feature I built into "Hemanth's Techhub Inventory System" to allow users to save their complete inventory data to an external CSV file.

When this procedure is activated, I first call `clear_screen` to ensure a focused display and then immediately inform the user of the ongoing process by displaying my `export_processing_msg`. The core of the export begins with creating the CSV file itself; I use DOS interrupt INT 21h with AH=3Ch, specifying `export_filename` and setting normal file attributes. I meticulously check for any errors during this creation, jumping to `export_error_exit_handling` if a problem occurs, and otherwise I store the returned `file_handle` to manage subsequent file operations. Once the file is created, I proceed to write the `export_header` (which contains the column names like "ID,Name,Price,Quantity") to the file, again employing INT 21h, AH=40h and verifying for any write errors.

After the header is in place, I initiate an `export_loop`, which I designed to iterate precisely nine times, once for each inventory item. Inside this loop, I systematically construct each line of the CSV. For the item ID, I take my `si` loop index, add '1' to convert it to a 1-based ID, convert it to an ASCII character, and call `write_char_to_file_internal` to write it. I then write a comma as a delimiter. To get the item's name, I efficiently retrieve its address from my `item_names_table` using the `si` index, then call `write_string_to_file_internal` to output it to the file, followed by another comma. I repeat this process for the item's price, fetching its address from `item_prices_table`. For the quantity, I directly access the `q_ItemX` byte variable corresponding to the current item using `q_Item0 + si` and call `write_byte_as_ascii_to_file_internal` to convert the numerical quantity to its ASCII representation before writing. Finally, I append a carriage return and a line feed (CRLF) to move to the next line in the CSV. I then increment my `si` index and allow the loop `export_loop` instruction to continue until all nine items have been written, ensuring all inventory data is meticulously transferred to the CSV file.

```

; Close file (INT 21h, AH=3Eh)
mov bx, [file_handle]
mov ah, 3Eh
int 21h
jc export_error_exit_handling ; Check for close error (unlikely)

; Success path: Display message and return to menu
DisplayMsg export_success ; Display success message
call ReturnToMenu ; Call ReturnToMenu as requested
jmp main_loop ; Jump directly to the main menu loop

export_error_exit_handling: ; Combined error handling for create, write, close
; Attempt to Close file if handle exists and an error occurred during creation/write
mov bx, [file_handle]
cmp bx, 0 ; Check if file handle was successfully obtained (0 means invalid or not yet opened)
je skip_close_on_error ; If not, skip closing
mov ah, 3Eh ; Close file
int 21h
skip_close_on_error:
DisplayMsg export_fail ; Display generic failure message

; On error, also return to menu
call ReturnToMenu
jmp main_loop ; Jump directly to the main menu loop

export_inventory ENDP

; Internal helper for writing a single character to the currently open file handle
; AL should contain the character to write
write_char_to_file_internal PROC
push ax
push bx
push cx
push dx
; AL already contains the character to write

mov [temp_char_buffer], al ; FIX: Store AL into the temporary buffer
; The next line is effectively line 663 in your original code after the fix.
; This approach avoids the 'Illegal indexing mode' error by using a static buffer.

mov bx, [file_handle] ; Get the file handle
mov ah, 40h ; Write to file
mov cx, 1 ; Write 1 byte
lea dx, temp_char_buffer ; DX points to the temporary buffer
int 21h

pop dx
pop cx
pop bx
pop ax
ret
write_char_to_file_internal ENDP

```

Figure 38: Source Code Screenshot 12

This final portion of my export_inventory procedure handles the critical steps of file finalization and comprehensive error management, complemented by my internal helper routines.

After I have successfully written all inventory data to the CSV file within the export loop, the next crucial step is to close the file. I achieve this by using DOS interrupt INT 21h with AH=3Eh, providing the file_handle that I obtained earlier. As a best practice, I always include an error check here (jc export_error_exit_handling), although errors during file closing are relatively rare. If the export is completely successful, I proudly display the export_success message to the user, confirming that their inventory data has been saved. Following this, I call ReturnToMenu to allow them to acknowledge the success, and then jmp main_loop to seamlessly bring them back to the main system menu, ready for their next action.

The `export_error_exit_handling` block is a robust safety mechanism I put in place to manage any issues that might arise during the file creation, writing, or closing phases. Here, I first attempt to close the file if a valid `file_handle` was ever obtained, preventing resource leaks. Regardless of the specific error, I display a generic `export_fail` message to the user, ensuring they are aware that the export did not complete as intended. Even in the event of an error, I still call `ReturnToMenu` and then `jmp main_loop`, ensuring a consistent user experience and preventing the program from crashing or getting stuck.

Finally, I have created `write_char_to_file_internal` as a dedicated helper procedure. This small but essential routine allows me to write a single character (passed in `AL`) to the currently open file handle. I make sure to preserve all necessary registers (push and pop) to maintain context. A key aspect of its design, and a fix I implemented during development, is storing the character temporarily in `temp_char_buffer` before writing it. This approach cleverly avoids an "Illegal indexing mode" error that might occur when trying to write single bytes directly, making my file writing operations more stable and reliable.

```

814 write_char_to_file_internal ENDP
815
816 ; Internal helper for writing a '$' terminated string to the currently open file handle
817 ; DX should already contain the string address
818 write_string_to_file_internal PROC
819     push ax
820     push bx
821     push cx
822     push dx
823     push si ; Preserve SI
824
825     mov bx, [file_handle] ; Get the file handle
826     ; Calculate string length
827     mov si, dx ; Copy string address to SI for length calculation
828     xor cx, cx ; Counter for length
829     string_length_loop:
830         cmp byte ptr [si], '$' ; Check for '$' terminator
831         je end_string_length
832         inc cx ; Increment length
833         inc si ; Move to next character
834         jmp string_length_loop
835     end_string_length:
836
837     mov ah, 40h ; Write to file
838     ; DX already holds the string pointer (original input DX)
839     int 21h ; Call DOS
840
841     pop si
842     pop dx
843     pop cx
844     pop bx
845     pop ax
846     ret
847 write_string_to_file_internal ENDP
848
849 ; Internal helper to convert a byte to ASCII string and write to file
850 ; BL should contain the byte (0-9)
851 write_byte_as_ascii_to_file_internal PROC
852     push ax
853     push bx
854     push cx
855     push dx
856
857     ; Handle single digit conversion easily (quantity is 0-9)
858     xor ah, ah ; Clear AH for division
859     mov al, bl ; Byte to AL (e.g., 4)
860     mov cl, 10 ; Divisor (for converting to digits)
861     div cl ; AL = quotient (tens digit), AH = remainder (units digit)
862
863     ; If quotient is not zero, write it (tens digit)
864     cmp al, 0
865     je skip_tens_digit
866     add al, '0' ; Convert quotient to ASCII
867     call write_char_to_file_internal
868 skip_tens_digit:
869     ; Write remainder (units digit)
870     mov al, ah ; Remainder in AL
871     add al, '0' ; Convert to ASCII
872     call write_char_to_file_internal
873
874     pop dx
875     pop cx
876     pop bx
877     pop ax
878     ret
879 write_byte_as_ascii_to_file_internal ENDP
880

```

Figure 39: Source Code Screenshot 13

This final set of procedures highlights some of the internal helper logic I developed to make the file export functionality of my "Hemanth's Techhub Inventory System" robust and flexible.

The `write_string_to_file_internal` procedure is a critical helper I designed for writing entire dollar-terminated strings to the currently open CSV file. When this procedure is called, it expects the string's address to be in DX. Before writing, I first meticulously calculate the exact length of the string by iterating through its bytes until I encounter the \$ terminator. This calculated length is essential for the DOS INT 21h, AH=40h function, which I then use to write the entire string to the file. Throughout this process, I ensure that all general-purpose registers are carefully preserved by pushing their values onto the stack at the beginning and restoring them at the end, preventing any unintended side effects on the calling code.

The `write_byte_as_ascii_to_file_internal` procedure is another specialized helper I created, specifically for converting a numerical byte (which in my inventory system represents a quantity from 0 to 9) into its ASCII character equivalent and then writing it to the file. This procedure expects the byte value to be in BL. While quantities in my system are single digits, I implemented a general division-by-10 logic: I divide the byte by 10 to separate any potential tens digit from the units digit. I then check if a tens digit exists (though for quantities 0-9, it will always be zero) and convert it to ASCII by adding '0' before writing it using my `write_char_to_file_internal` helper. Finally, I convert the units digit (remainder) to ASCII and write it as well. This ensures that any quantity, even single-digit ones, is correctly formatted as a readable character in the exported CSV file. As with all my helper procedures, I diligently preserve all registers to maintain the integrity of the main program flow.

```

882 ;Confirmation for Exit
883 exit_confirmed:
884
885     DisplayMsg Exit
886     mov ah,01h
887     int 21h
888
889     cmp al,'n'
890     je main_loop
891
892     cmp al,'y'
893     call clear_screen
894     DisplayMsg TyMsg
895     je exit_main
896
897 ;Prompt User to enter Menu
898 ReturnToMenu:
899     DisplayMsg EnterRet
900     mov ah, 01h          ; Read user character input
901     int 21h
902     ;Check if the Enter key pressed
903     cmp al, 0Dh
904     ;If not, prompt again
905     jne ReturnToMenu
906
907     ;If yes, return to the main loop
908     ret
909
910 ;Clear Screen
911 clear_screen:
912     mov ah, 06h
913     mov al, 0
914     mov bh, 07h
915     mov cx, 0
916     mov dx, 184Fh
917     int 10h
918     ret
919
920 ;End Program
921 exit_main:
922     mov ah,4Ch
923     int 21h
924
925 MAIN ENDP
926 END MAIN

```

Figure 40: Source Code Screenshot 14

This final set of procedures brings my "Hemanth's Techhub Inventory System" to a complete and user-friendly close.

The exit_confirmed procedure is my way of gracefully handling the user's decision to leave the program. When this option is selected, I first display my Exit prompt, asking the user to confirm their intention with a 'y' or 'n'. I then read their input. If they enter 'n', I immediately direct them back to the main_loop, allowing them to continue using the system. However, if 'y' is entered, I first call clear_screen for a clean exit presentation, then display my heartfelt TyMsg (Thank You message), and finally je exit_main to terminate the program.

The ReturnToMenu procedure is a small but important utility I designed for user convenience. Its purpose is to pause execution and await user acknowledgment before returning to a menu. I display

the EnterRet message, prompting the user to press the Enter key. I then continuously read keyboard input, and only when the Enter key (ASCII 0Dh) is detected does the procedure return, ensuring the user has ample time to review any displayed information before proceeding.

The clear_screen procedure is a vital visual component of my system. This procedure is my standard way to refresh the console display, ensuring that each new menu or inventory view is presented on a clean screen, free from previous clutter. I achieve this by utilizing the BIOS INT 10h interrupt with AH=06h, specifying the entire screen area to scroll (effectively clearing it) and setting a default background color.

Lastly, the exit_main procedure is the definitive endpoint of my program. This is the routine I call to formally terminate the entire application, utilizing INT 21h, AH=4Ch to return control to the operating system. Following this, MAIN ENDP marks the end of my MAIN procedure, and END MAIN signifies the absolute end of my entire assembly source code, indicating where the program execution should begin.

6.0 Conclusion

Throughout this assignment, I embarked on a multifaceted exploration of low-level programming and cybersecurity principles, encompassing both the development of a practical inventory management system in assembly language and the challenging process of reverse engineering a binary from the PicoCTF challenge. This journey has not only solidified my understanding of fundamental computer science concepts but also provided invaluable hands-on experience in areas crucial for modern computing and security.

My development of the "Hemanth's Techhub Inventory System" in assembly language served as a cornerstone of this project. Through this endeavor, I gained a profound appreciation for the intricacies of direct hardware interaction and memory management. I meticulously crafted data structures to store inventory items, their quantities, and prices, and designed efficient routines for displaying this information, managing stock levels through selling and restocking functionalities, and even exporting data to a CSV file. The process of writing, debugging, and refining macros like `DisplayMsg`, `SellItem`, and `RestockItem` provided practical insights into modular programming at the assembly level, emphasizing the importance of efficient code organization. Implementing features such as robust user authentication, including password and username validation, further underscored the foundational elements of secure application design, even in a low-level environment. The ability to directly control program flow, manage registers, and interact with DOS interrupts like `INT 21h` for input/output and file operations offered a unique perspective on how software interacts with the operating system, a detail often abstracted away in higher-level languages.

Complementing my development work, the "Bbbloat" reverse engineering challenge from PicoCTF presented a compelling practical application of analytical and problem-solving skills. Utilizing Ghidra, a powerful reverse engineering framework, I systematically dissected the provided binary. My initial approach involved mapping out the program's functions and identifying the main entry point. The challenge's namesake, "bloat," became evident as I navigated through various functions, some seemingly redundant, designed to obscure the core logic. A critical breakthrough occurred when I discovered `FUN_00101307`. Through careful analysis of its decompiled C code, I identified a `printf` prompt for a "favorite number" and, more importantly, a direct comparison of user input against a hardcoded hexadecimal value, `0x86187`. This revelation was pivotal. I then used a hexadecimal-to-decimal converter to obtain the required numerical input, `549255`, which allowed me to bypass the program's internal check. Finally, after ensuring the binary was executable on the PicoCTF webshell, I executed the program, provided the calculated decimal input, and successfully retrieved the flag.

In conclusion, this assignment has been an immersive experience that bridged theoretical knowledge with practical application. From constructing a functional system from the ground up in assembly language to deconstructing a complex binary to extract hidden information, I have honed essential skills in software architecture, debugging, security analysis, and critical thinking. The lessons learned about optimizing code, ensuring data integrity, and navigating obfuscated program logic are invaluable and will undoubtedly serve as a strong foundation for future endeavors in both software development and cybersecurity.

7.0 References

CliffsNotes. (2021). *Mastering Turbo Assembler: A guide to assembly language*.

<https://www.cliffsnotes.com/study-notes/23032680>

Dahlquist, J. (2021). *Assembly language fundamentals*. California State University, Sacramento.

<https://ecs-pw->

[facweb.ecs.csus.edu/~dahlquid/eee174/S2005/Lecture06/174lecture06_files/asmlangbasics_3.pdf](https://ecs-pw-facweb.ecs.csus.edu/~dahlquid/eee174/S2005/Lecture06/174lecture06_files/asmlangbasics_3.pdf)

Microsoft. (2023). *Microsoft macro assembler reference*. [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference)

[us/cpp/assembler/masm/microsoft-macro-assembler-reference](https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference)

Netwide Assembler Project. (2022). *NASM manual: Memory and operand syntax*. [https://ece-](https://ece-research.unm.edu/jimp/310/nasm/nasmdoc2.html)

[research.unm.edu/jimp/310/nasm/nasmdoc2.html](https://ece-research.unm.edu/jimp/310/nasm/nasmdoc2.html)

Netwide Assembler Project. (2023). *NASM operand-size prefixes: o16 and o32*.

<https://posix.nl/linuxassembly/nasmdochtml/nasmdoca.html>

OSDev Community. (2023, November 28). *TASM advantages: Ideal mode and macros*.

<https://wiki.osdev.org/TASM>

Tutorials Point. (2021). *Assembly language tutorial*.

https://www.tutorialspoint.com/assembly_programming/assembly_tutorial.pdf

Worcester Polytechnic Institute. (2021). *Using the Turbo Assembler tools*.

http://web.cs.wpi.edu/~jburge/courses/a01/cs2011/tasm_instructions.html

AllAboutCircuits. (2022). *Is assembly language still relevant for embedded software*

development? <https://www.allaboutcircuits.com/technical-articles/is-assembly-language-relevant-for-embedded-software-development/>

AllAboutCircuits Forum. (2023). *Using assembly in real-time embedded firmware*.
<https://forum.allaboutcircuits.com/threads/using-assembly-in-real-time-firmware.193881/>

Bilokon, P. A., & Lucuta, M. (2023). *C++ design patterns for low-latency applications including high-frequency trading*. arXiv. <https://arxiv.org/abs/2309.04259>

DragonZap. (2023). *Creating a simple bootloader in assembly language: A beginner's guide*.
<https://dragonzap.com/blog/simple-bootloader-assembly-language>

Embedded.com. (2023). *Is assembly language obsolete?* <https://www.embedded.com/is-assembly-language-obsolete/>

Frosnerd. (2020). *Writing efficient ISRs in assembly for microcontrollers*.
<https://frosnerd.medium.com/efficient-interrupt-service-routines-in-assembly-1b2f89f5a239>

MoldStud Research. (2025). *Why assembly language still matters in 2025: From bootloaders to trading engines*. <https://moldstud.com/blog/why-assembly-language-still-matters>

The Pi Guy. (2024, January 15). *Assembly languages in modern embedded systems: Trends and applications*. https://the-pi-guy.com/blog/assembly_languages_in_modern_embedded_systems_trends_and_applications/

UEFI Specification Forum. (2025). *Unified Extensible Firmware Interface (UEFI) specification, version 2.10*. <https://uefi.org/specifications>

UNC COMP730. (2025). *COMP730 Embedded Systems Lab Manual – Week 3: Writing a bootloader in assembly*. University of North Carolina.
<https://comp730lab.unc.edu/2025/bootloader-lab-guide>

Alam, S. (2022). *MAIL: Malware Analysis Intermediate Language*. arXiv.
<https://arxiv.org/abs/2211.03068>

BugBustersUnited. (n.d.). *Assembly language: The foundation of reverse engineering and exploits*. <https://bugbustersunited.com/assembly-language-the-reverse-engineerings/>

Kenji. (2024). *Mastering reverse engineering with Ghidra: A comprehensive guide*. Medium. <https://medium.com/@kenjikun/mastering-reverse-engineering-with-ghidra-a-comprehensive-guide-377fe5dc8435>

Lepide. (2024, December 24). *What is Ghidra and how to use it to reverse engineer malware*. Lepide. <https://www.lepide.com/blog/what-is-ghidra-and-how-to-use-it/>

Puckett, T. (2022, October 13). *Malware analysis and introduction to assembly language*. IBM Training Blog. <https://www.ibm.com/blogs/ibm-training/malware-analysis-and-introduction-to-assembly-language/>

Varonis. (2023, April 6). *How to use Ghidra to reverse engineer malware*. Varonis Blog. <https://www.varonis.com/blog/how-to-use-ghidra>

8.0 Appendices

